

<https://www.youtube.com/watch?v=fHFOANOHwh8>

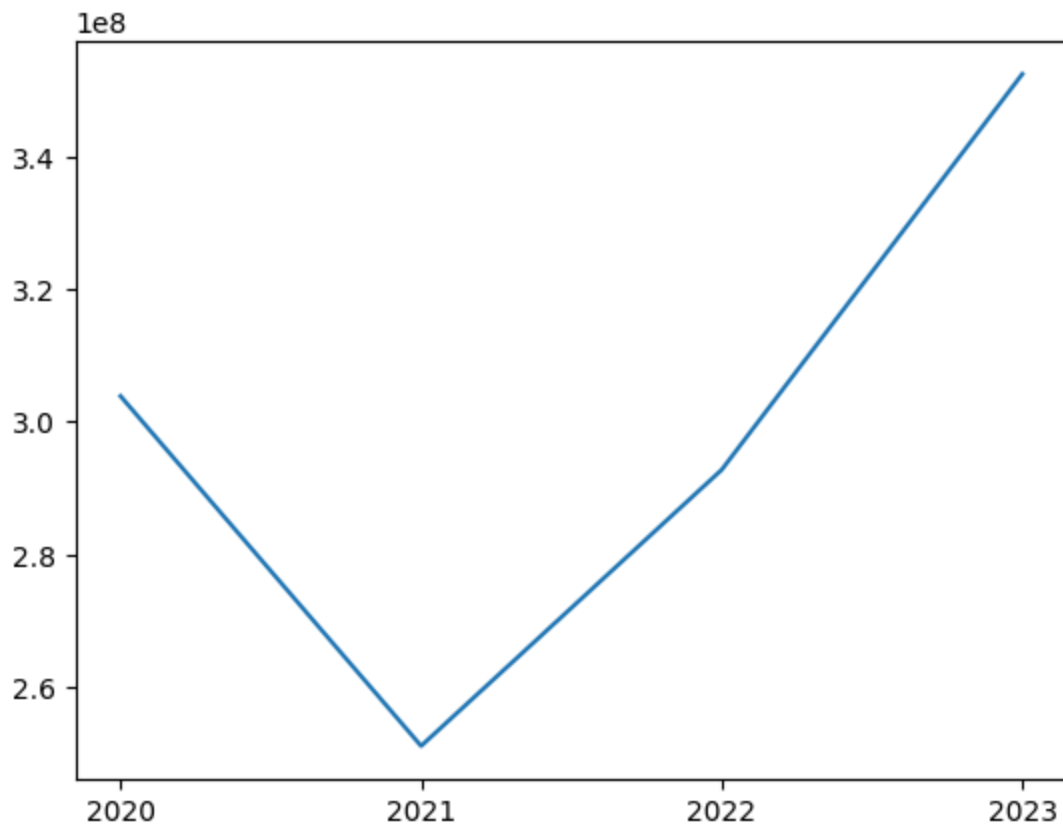
```
In [ ]: # To install matplotlib
! pip install matplotlib
```

```
Requirement already satisfied: matplotlib in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (3.9.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (1.2.1)
Requirement already satisfied: cyclor>=0.10 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (1.4.5)
Requirement already satisfied: numpy>=1.23 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (24.0)
Requirement already satisfied: pillow>=8 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (10.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\dilip\appdata\local\programs\python\python310\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

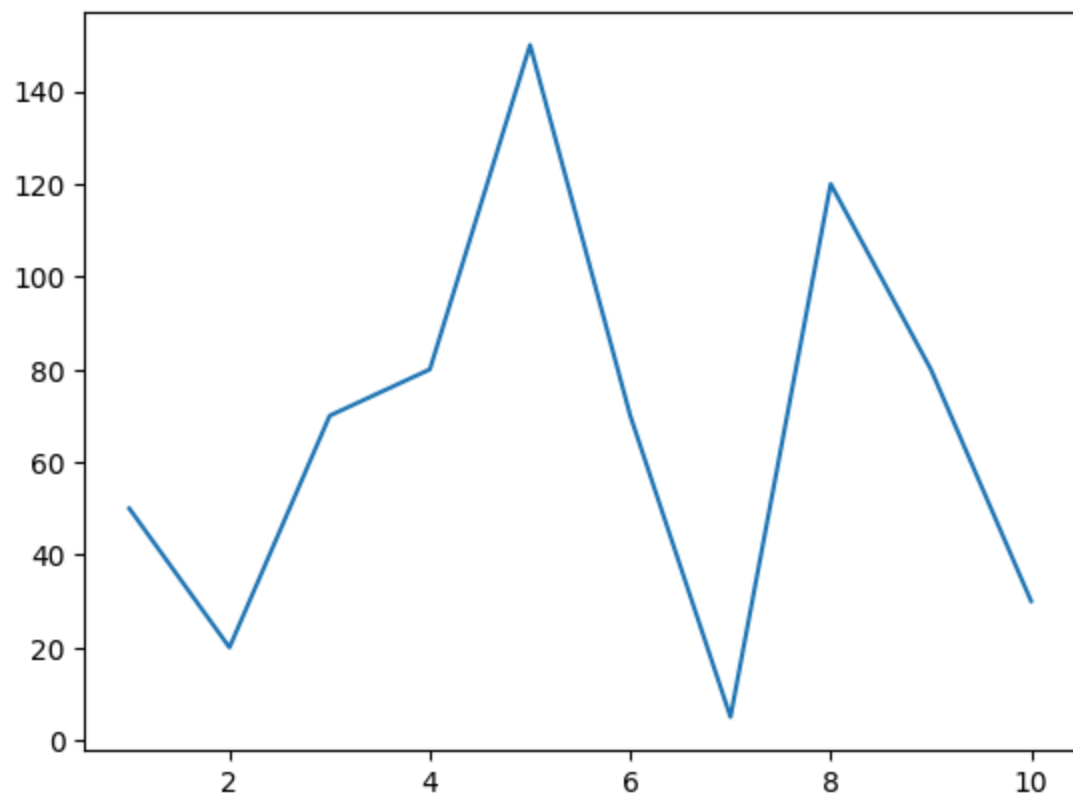
```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [8]: a =[352583000,292755000,251002000,303888000]

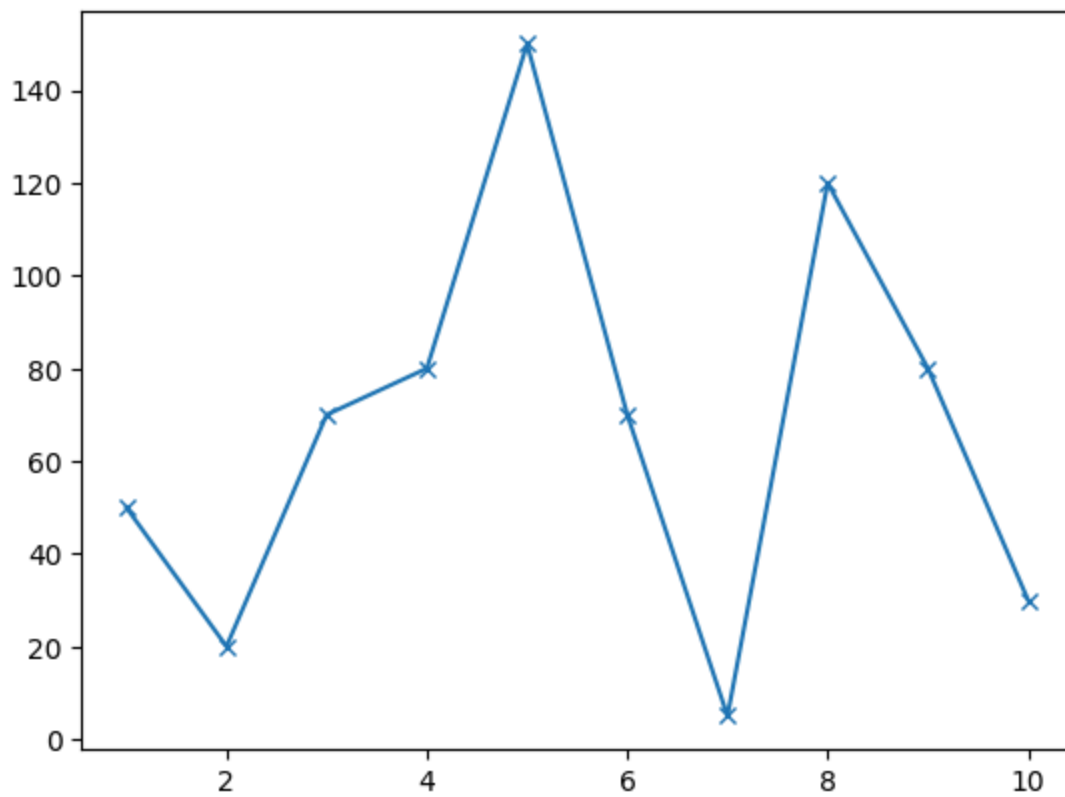
# xpoints = np.arange(1,11)
# ypoints = np.array([50,20,70,80, 150, 70, 5, 120, 80, 30])
plt.plot([2023,2022,2021,2020], a)
plt.xticks([2023,2022,2021,2020])
plt.show()
```



```
In [10]: xpoints = np.arange(1,11)
ypoints = np.array([50,20,70,80, 150, 70, 5, 120, 80, 30])
plt.plot(xpoints, ypoints)
plt.show()
```



```
In [ ]: plt.plot(xpoints,ypoints, marker = 'x')
plt.show()
```

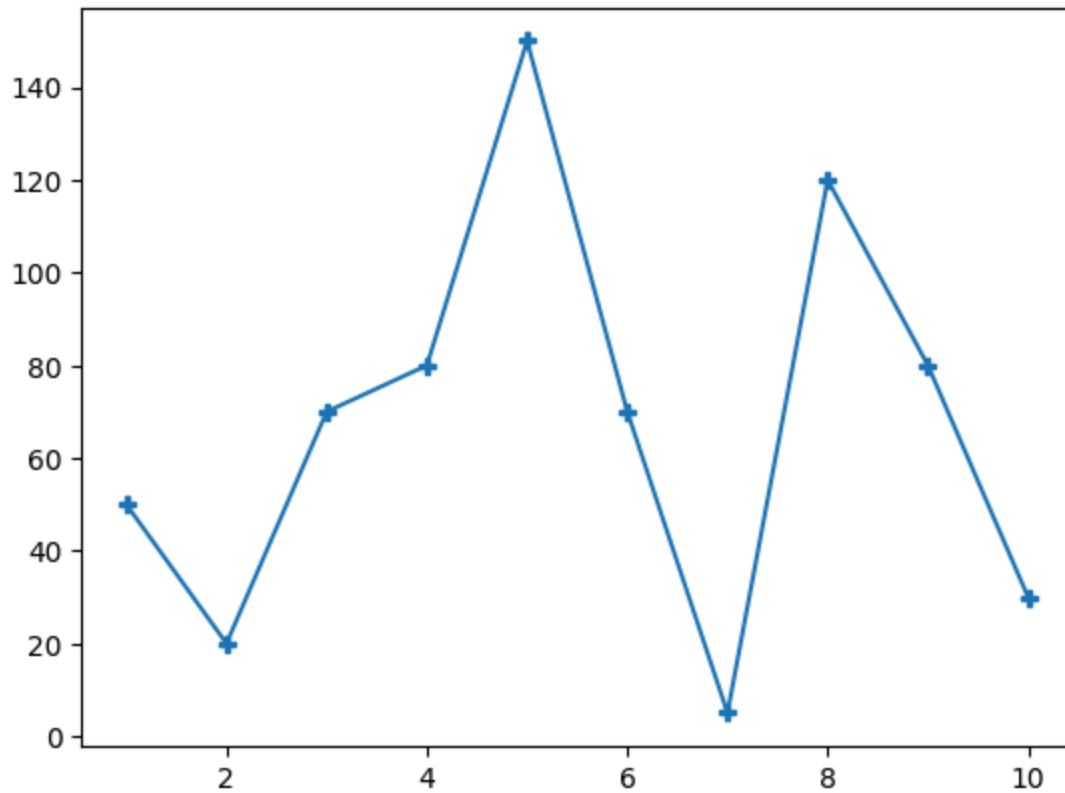


Marker Description

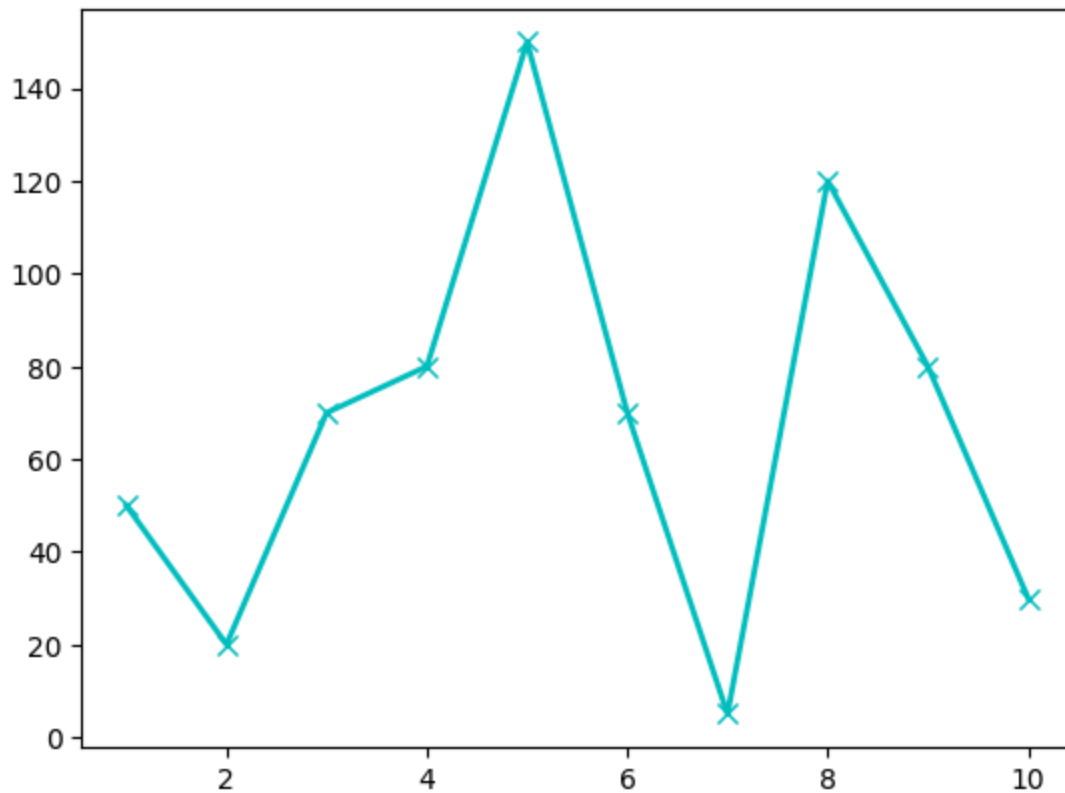
'o'	Circle
'*'	Star
'.'	Point
','	Pixel
'x'	X
'X'	X (filled)
'+'	Plus
'p'	Plus (filled)
's'	Square
'D'	Diamond
'd'	Diamond (thin)
'p'	Pentagon
'H'	Hexagon
'h'	Hexagon
'v'	Triangle Down
'^'	Triangle Up
'<'	Triangle Left
'>'	Triangle Right
'1'	Tri Down
'2'	Tri Up
'3'	Tri Left
'4'	Tri Right

'|' Vline
'_' Hline

```
In [ ]: plt.plot(xpoints,ypoints, marker = 'P')  
plt.show()
```



```
In [ ]: plt.plot(xpoints,ypoints, 'x-c',linewidth=2, markersize=7) # (data, dot style, line  
plt.show()
```



'-' or 'solid' solid line

'--' or 'dashed' dashed line

'-.' or 'dashdot' dash-dotted line

':' or 'dotted' dotted line

'None' draw nothing

' ' draw nothing

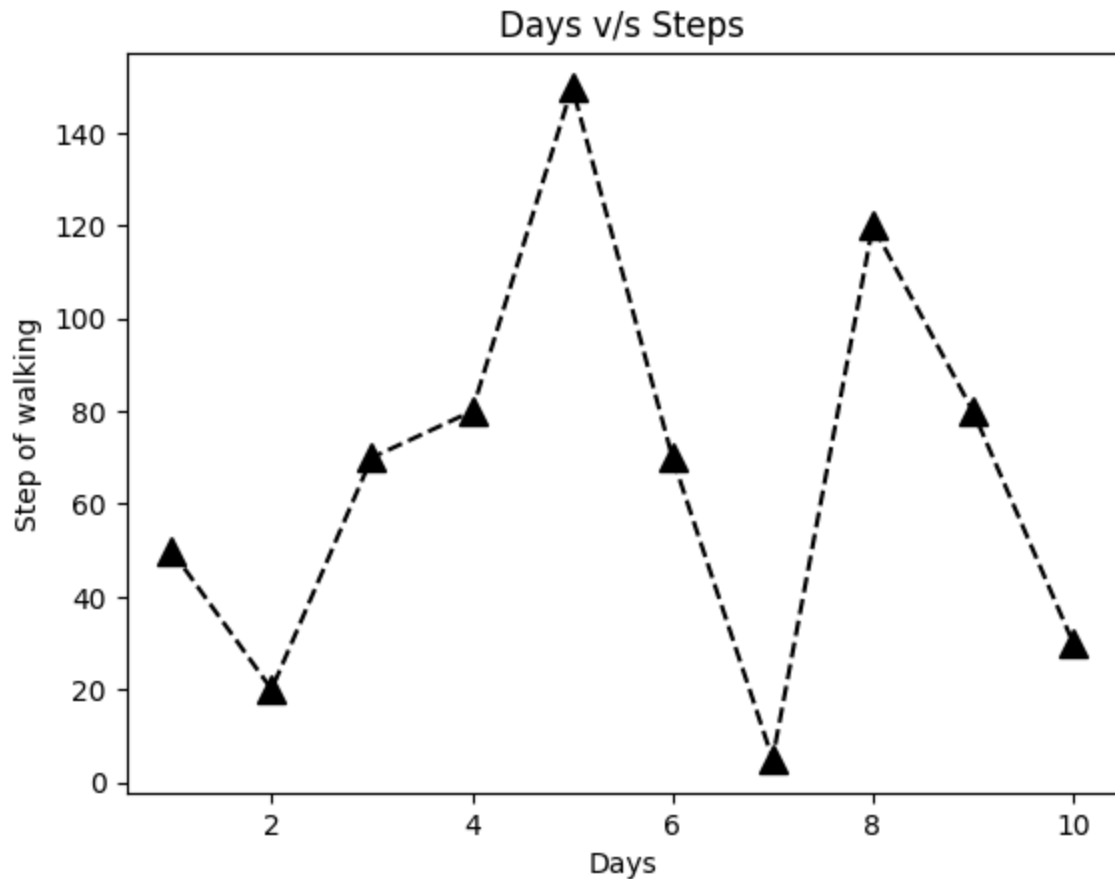
'' draw nothing

Color Syntax	Description
'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

```
In [12]: plt.plot(xpoints, ypoints, '^--k', ms = 10) # (data, dot style, line style, color c
```

```
plt.xlabel('Days')
plt.ylabel('Step of walking')
plt.title('Days v/s Steps')

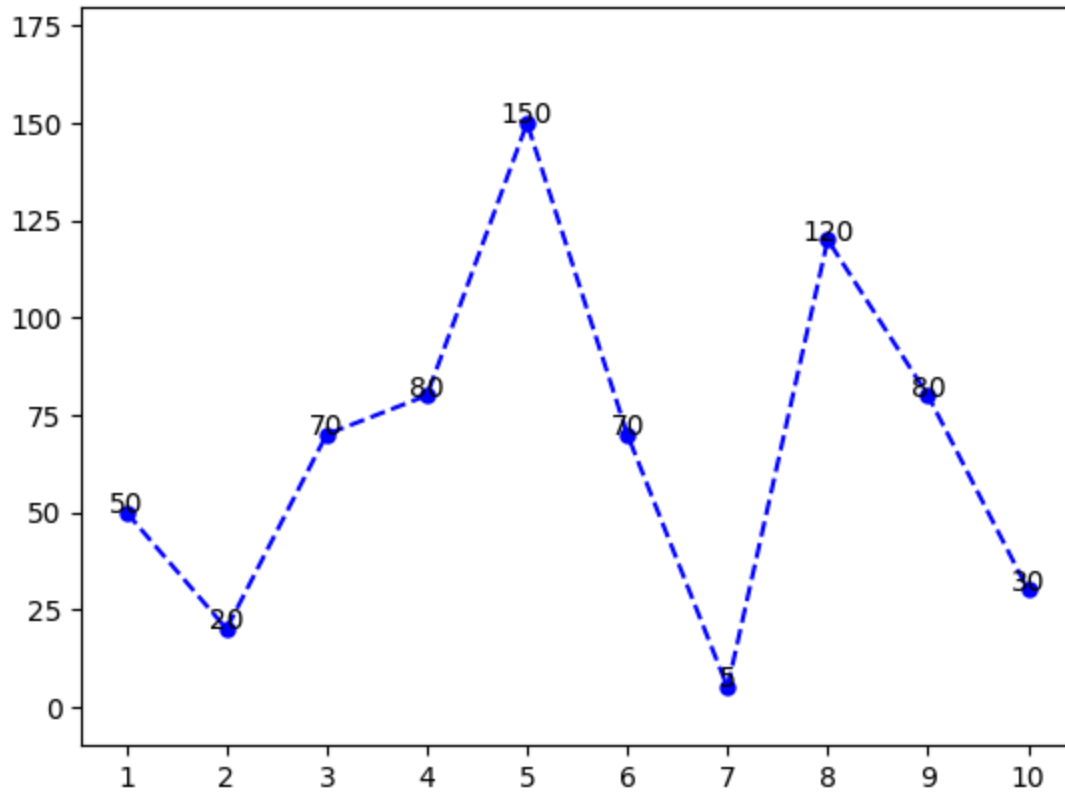
plt.show()
```



```
In [16]: fig = plt.figure()
ax = fig.add_subplot(111)
# values = [7, 57, 121, 192, 123, 240, 546]
# Labels = ['1950s', '1960s', '1970s', '1980s', '1990s', '2000s', '2010s']

xpoints = np.arange(1,11)
ypoints = np.array([50,20,70,80, 150, 70, 5, 120, 80, 30])
plt.plot(xpoints, ypoints, 'b--o', ms = 5) # Plotting data
plt.xticks(xpoints) # Redefining x-axis labels
# ax.text(1,50, '999990')
for i, v in enumerate(ypoints,1):
    ax.text(i, v, "%d" %v, ha='center')
plt.ylim(-10, 180)
```

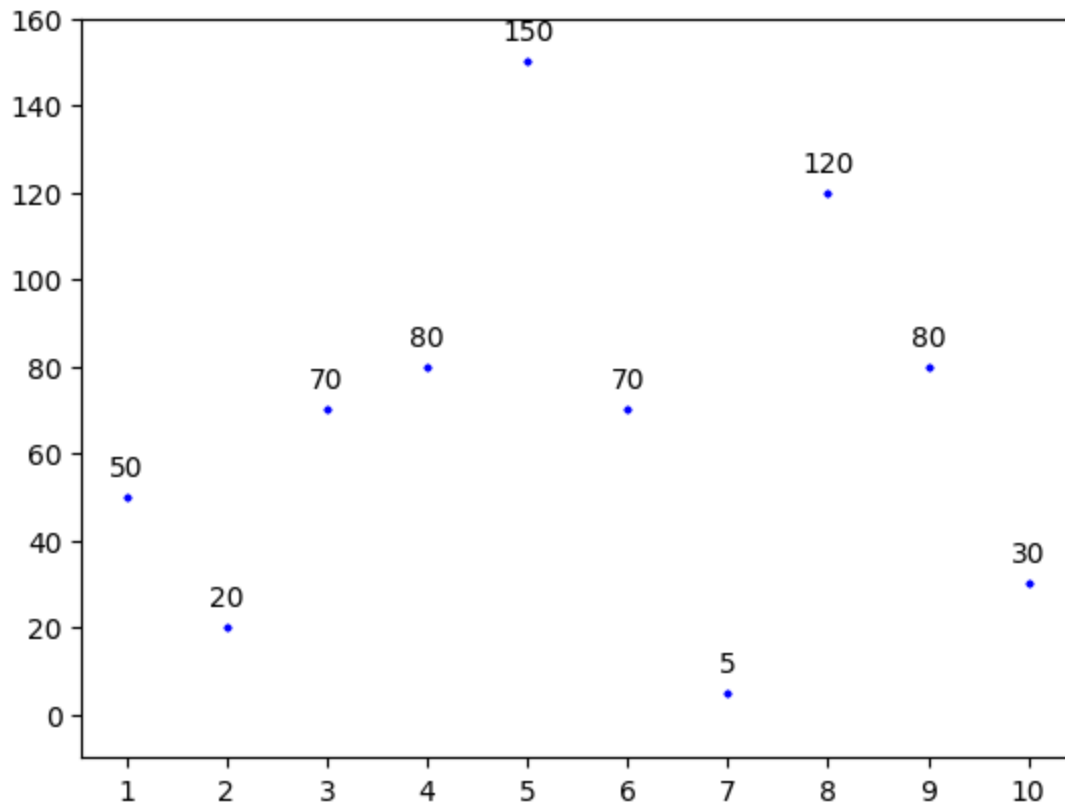
Out[16]: (-10.0, 180.0)



```
In [21]: fig = plt.figure()
ax = fig.add_subplot(111)
# values = [7, 57, 121, 192, 123, 240, 546]
# labels = ['1950s', '1960s', '1970s', '1980s', '1990s', '2000s', '2010s']

xpoints = np.arange(1,11)
ypoints = np.array([50,20,70,80, 150, 70, 5, 120, 80, 30])
plt.plot(xpoints, ypoints, 'bP', ms = 2) # Plotting data
plt.xticks(xpoints) # Redefining x-axis labels
# ax.text(1,50, '999990')
for i, v in enumerate(ypoints):
    ax.text(i+1, v+5, "%d" %v, ha='center')
plt.ylim(-10, 160)
```

Out[21]: (-10.0, 160.0)

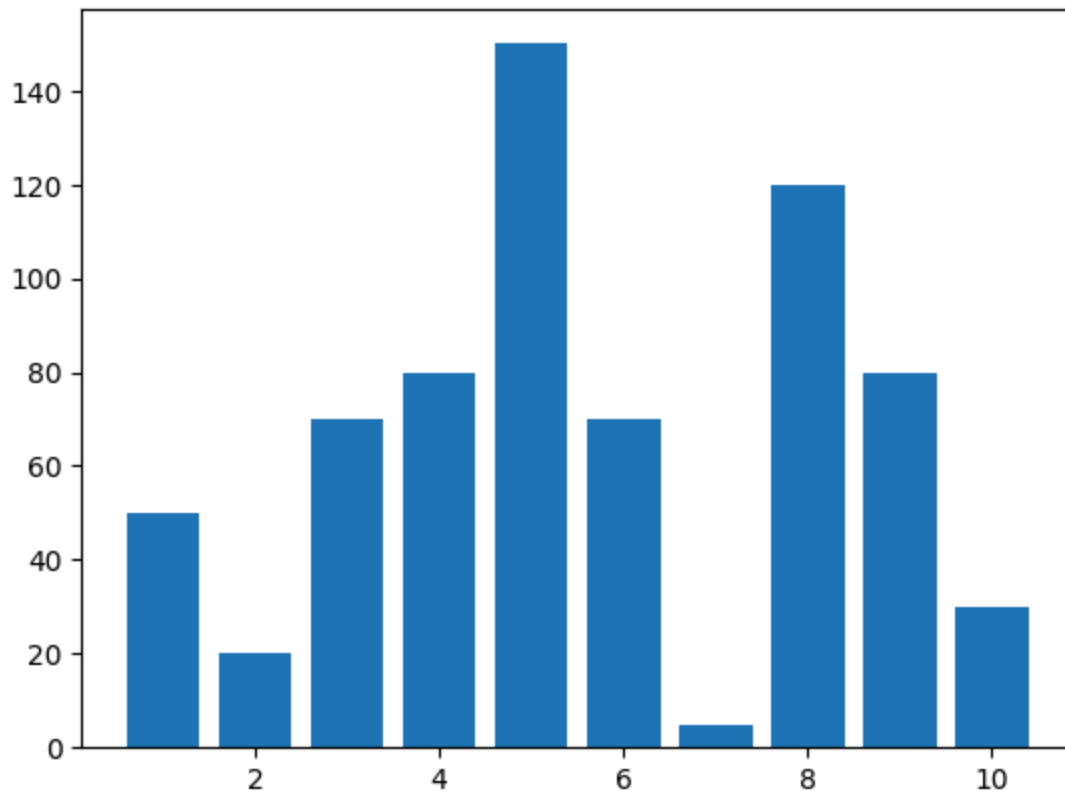


```
plt.plot(ypoints, '^--m', ms = 20) # (data, dot style, line style, color code)
```

```
plt.xlabel('Days') plt.ylabel('Step of walking')
```

```
plt.show()
```

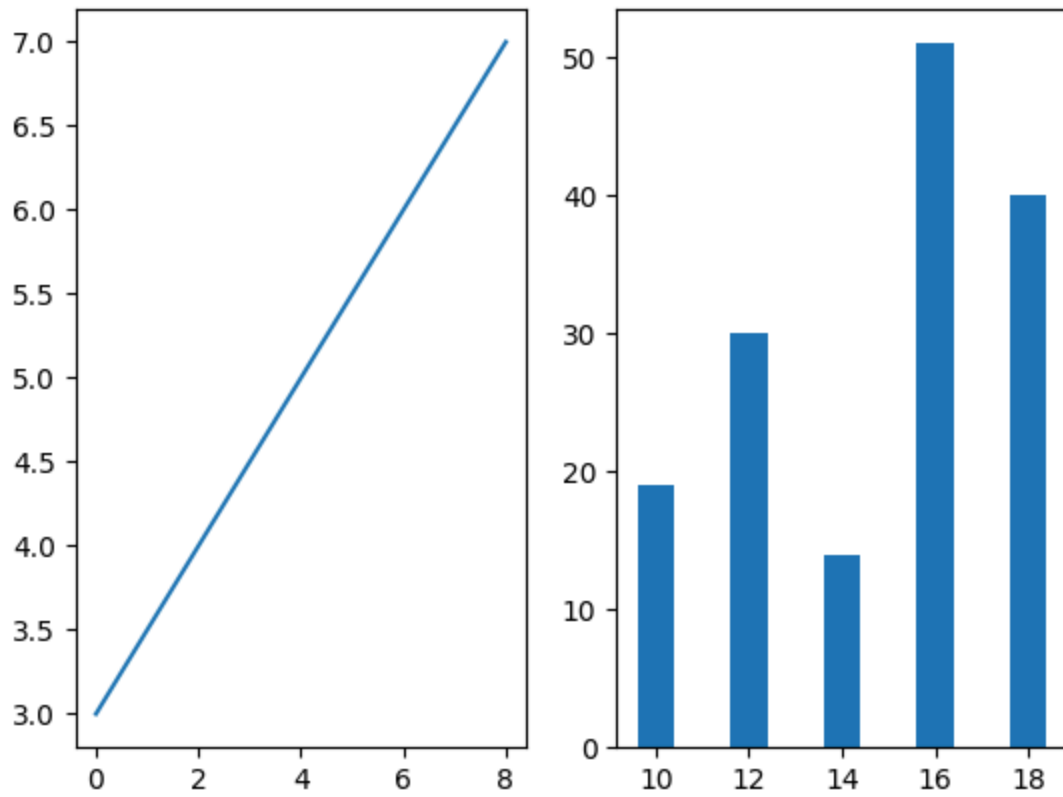
```
In [22]: xpoints = np.arange(1,11)
ypoints = np.array([50,20,70,80, 150, 70, 5, 120, 80, 30])
plt.bar(xpoints, ypoints)
plt.show()
```



```
In [24]: x = np.arange(0,10,2)
y = np.arange(3,8)

plt.subplot(1,2,1)
plt.plot(x,y)
x1 = np.arange(10,20,2)
y1 = np.array([19,30,14,51,40])

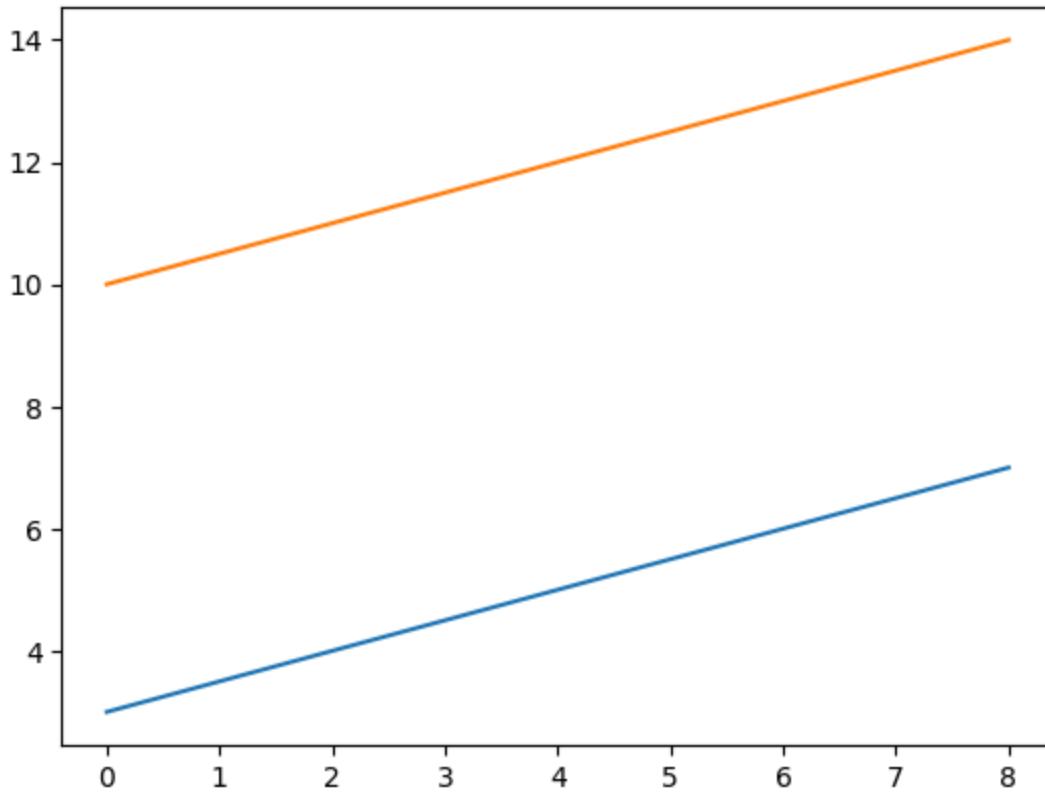
plt.subplot(1,2,2)
plt.bar(x1,y1)
plt.show()
```



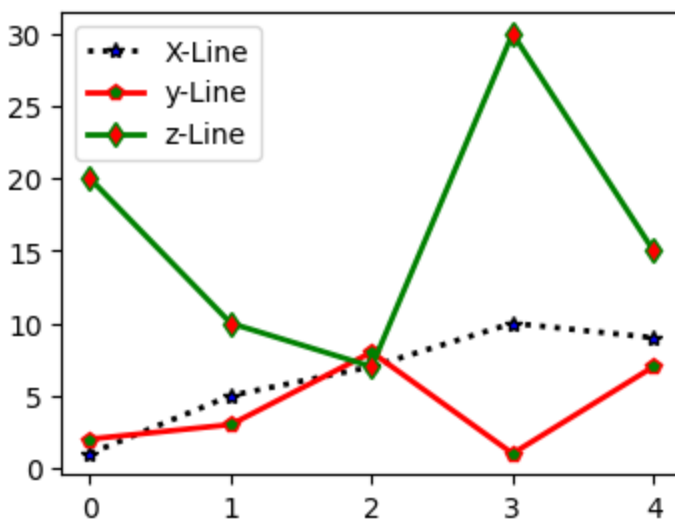
```
In [25]: x = np.arange(0,10,2)
y = np.arange(3,8)

# plt.subplot(1,2,1)
plt.plot(x,y)
x1 = np.arange(10,20,2)
y1 = np.arange(10,15)

# plt.subplot(1,2,2)
plt.plot(x,y1)
plt.show()
```



```
In [35]: plt.figure(figsize=(4,3)) # x - axis , y - axis
x = np.array([1,5,7,10,9])
y = np.array([2,3,8,1,7])
z = np.array([20,10,7,30,15])
plt.plot(x,'*:k', lw = 2, mfc = 'b', label = 'X-Line')
plt.plot(y,'p-r', lw = 2, mfc = 'g', label = 'y-Line')
plt.plot(z,'d-g', lw = 2, mfc = 'r', label = 'z-Line')
plt.legend()
plt.show()
```

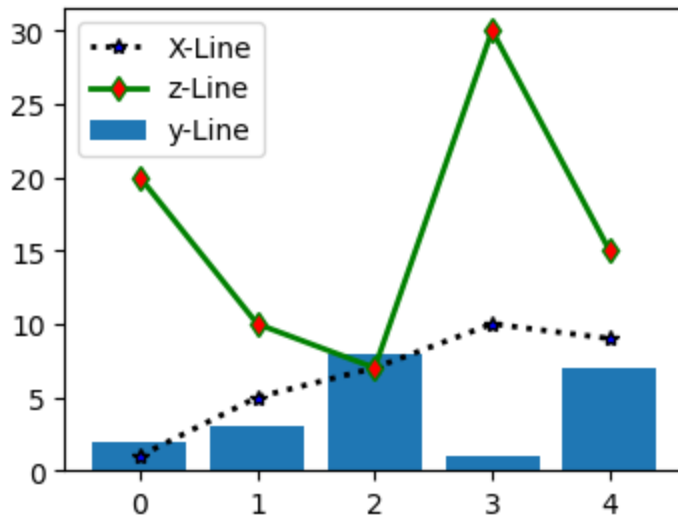


```
In [38]: plt.figure(figsize=(4,3)) # x - axis , y - axis
x = np.array([1,5,7,10,9])
x1 = np.array([0,1,2,3,4])
y = np.array([2,3,8,1,7])
```

```

z = np.array([20,10,7,30,15])
plt.plot(x,'*k', lw = 2, mfc = 'b', label = 'X-Line')
plt.bar(x1,y,label = 'y-Line')
plt.plot(z,'d-g', lw = 2, mfc = 'r', label = 'z-Line')
plt.legend()
plt.show()

```



```

In [39]: from google.colab import drive
drive.mount('/content/MyDrive')

```

Mounted at /content/MyDrive

```

In [40]: pwd

```

```

Out[40]: '/content'

```

```

In [41]: cd '/content/MyDrive/MyDrive/01_Learners_files/Aditi_Ayushi_Divya_Falguni_Manthan/A
/content/MyDrive/MyDrive/01_Learners_files/Aditi_Ayushi_Divya_Falguni_Manthan/AADM_E
DA

```

Univariant Graph (Univariant Analysis)

```

In [43]: df = pd.read_csv('brain_stroke.csv')
df

```

Out[43]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
0	Male	67.0	0	1	Yes	Private	Urban
1	Male	80.0	0	1	Yes	Private	Rural
2	Female	49.0	0	0	Yes	Private	Urban
3	Female	79.0	1	0	Yes	Self-employed	Rural
4	Male	81.0	0	0	Yes	Private	Urban
...	...	...	...	...	...	...	...
4976	Male	41.0	0	0	No	Private	Rural
4977	Male	40.0	0	0	Yes	Private	Urban
4978	Female	45.0	1	0	Yes	Govt_job	Rural
4979	Male	40.0	0	0	Yes	Private	Rural
4980	Female	80.0	1	0	Yes	Private	Urban

4981 rows × 11 columns

In [44]: `df.shape`

Out[44]: (4981, 11)

```
In [45]: # Basic EDA --> Get the information of dataframe
# How many row and columns
print('Rows = ',df.shape[0],'\nColumns = ',df.shape[1])
```

```
Rows = 4981
Columns = 11
```

In []:

```
In [ ]: # Check the basic information of data i.e. data type of columns, column's name
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4981 entries, 0 to 4980
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 4981 non-null   object
1   age                   4981 non-null   float64
2   hypertension           4981 non-null   int64
3   heart_disease         4981 non-null   int64
4   ever_married          4981 non-null   object
5   work_type             4981 non-null   object
6   Residence_type        4981 non-null   object
7   avg_glucose_level     4981 non-null   float64
8   bmi                   4981 non-null   float64
9   smoking_status        4981 non-null   object
10  stroke                 4981 non-null   int64
dtypes: float64(3), int64(3), object(5)
memory usage: 428.2+ KB

```

```

In [46]: # to check the statistical values of data
df.describe()

```

```

Out[46]:
```

	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	4981.000000	4981.000000	4981.000000	4981.000000	4981.000000	4981.000000
mean	43.419859	0.096165	0.055210	105.943562	28.498173	0.049789
std	22.662755	0.294848	0.228412	45.075373	6.790464	0.217531
min	0.080000	0.000000	0.000000	55.120000	14.000000	0.000000
25%	25.000000	0.000000	0.000000	77.230000	23.700000	0.000000
50%	45.000000	0.000000	0.000000	91.850000	28.100000	0.000000
75%	61.000000	0.000000	0.000000	113.860000	32.600000	0.000000
max	82.000000	1.000000	1.000000	271.740000	48.900000	1.000000

```

In [47]: df['hypertension'].value_counts()

```

```

Out[47]:
```

	count
hypertension	
0	4502
1	479

dtype: int64

```

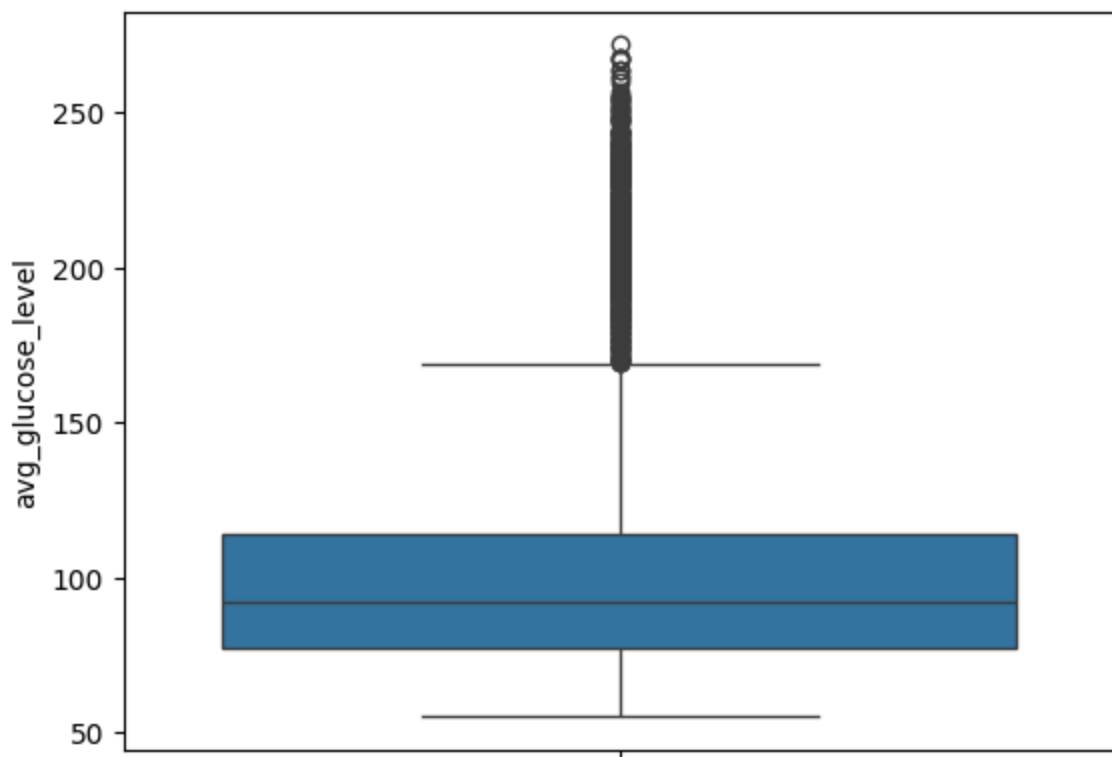
In [ ]: df.describe().T

```

	count	mean	std	min	25%	50%	75%	max
age	4981.0	43.419859	22.662755	0.08	25.00	45.00	61.00	82.00
hypertension	4981.0	0.096165	0.294848	0.00	0.00	0.00	0.00	1.00
heart_disease	4981.0	0.055210	0.228412	0.00	0.00	0.00	0.00	1.00
avg_glucose_level	4981.0	105.943562	45.075373	55.12	77.23	91.85	113.86	271.74
bmi	4981.0	28.498173	6.790464	14.00	23.70	28.10	32.60	48.90
stroke	4981.0	0.049789	0.217531	0.00	0.00	0.00	0.00	1.00

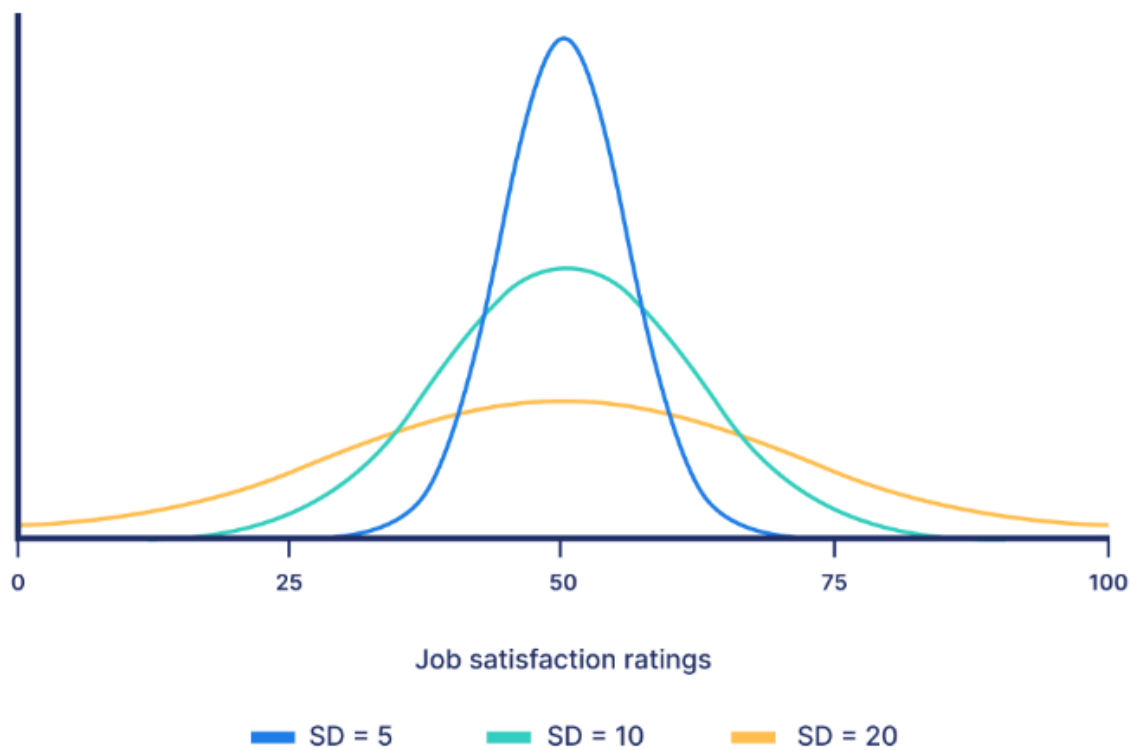
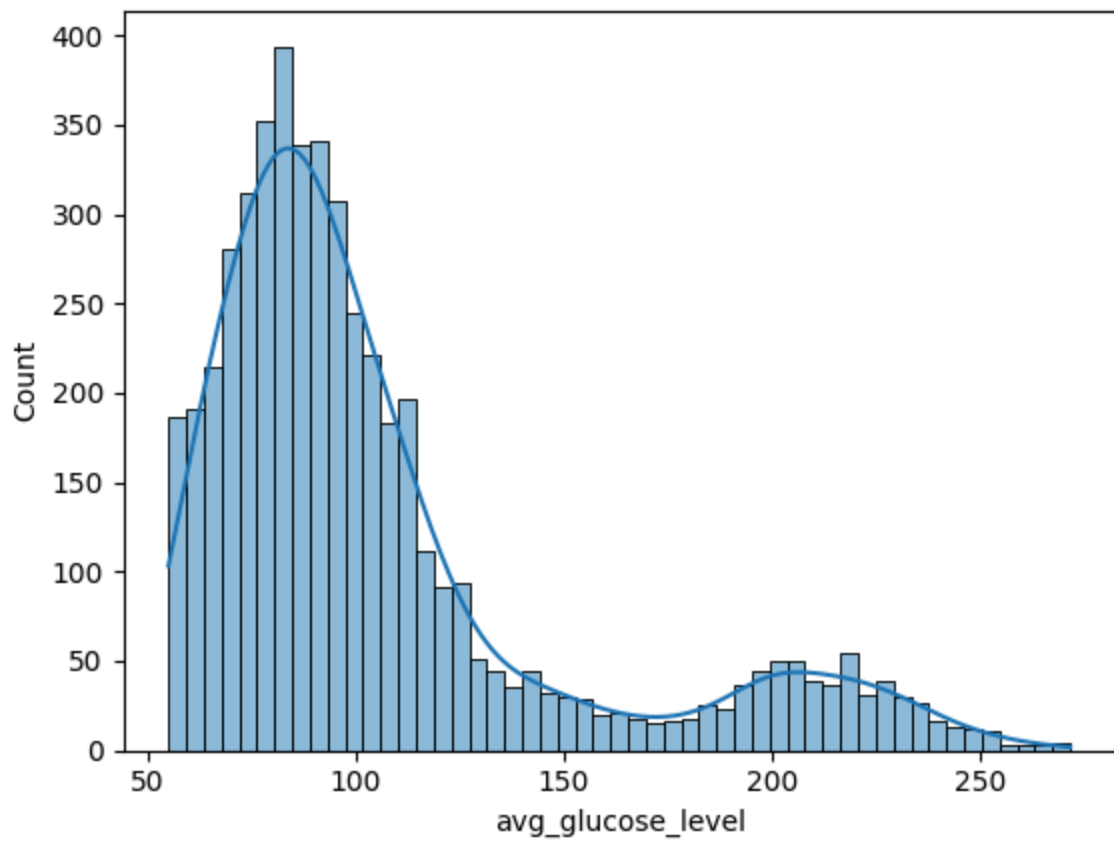
```
In [ ]: sns.boxplot(df['avg_glucose_level'])
```

```
Out[ ]: <Axes: ylabel='avg_glucose_level'>
```

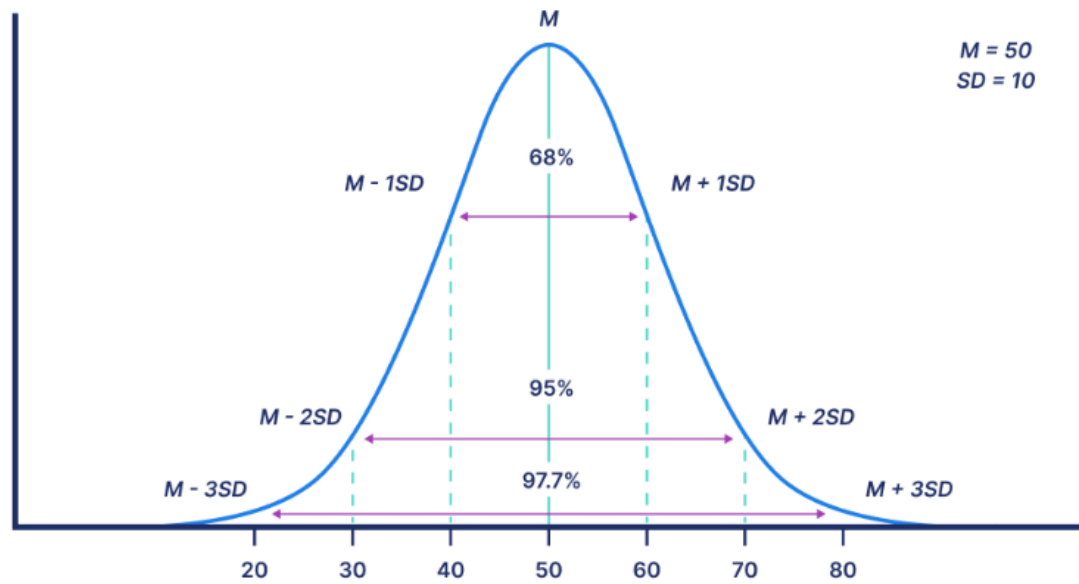


```
In [ ]: sns.histplot(df['avg_glucose_level'], kde = True)
```

```
Out[ ]: <Axes: xlabel='avg_glucose_level', ylabel='Count'>
```



Standard deviations in a normal distribution



```
In [ ]: df.describe(include = 'object')
```

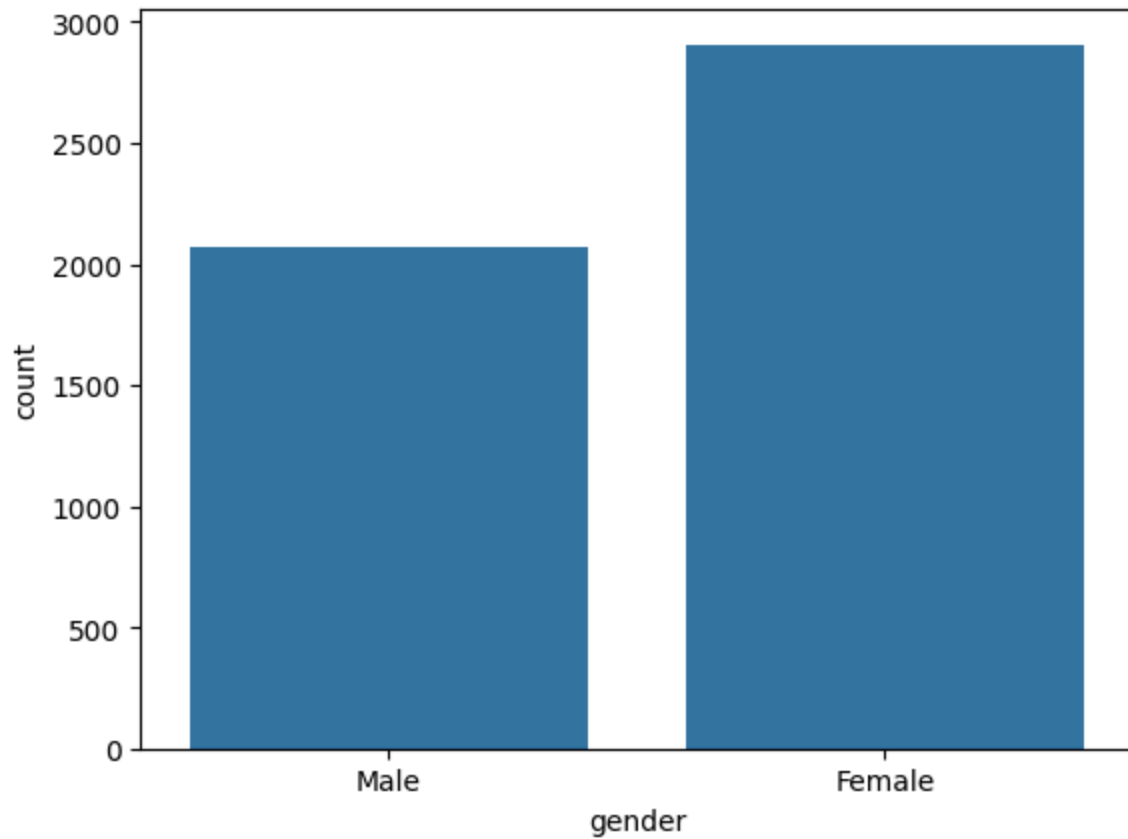
```
Out[ ]:
```

	gender	ever_married	work_type	Residence_type	smoking_status
count	4981	4981	4981	4981	4981
unique	2	2	4	2	4
top	Female	Yes	Private	Urban	never smoked
freq	2907	3280	2860	2532	1838

```
In [ ]: print(df['gender'].value_counts())
sns.countplot(x = df['gender'])
```

```
gender
Female    2907
Male      2074
Name: count, dtype: int64
```

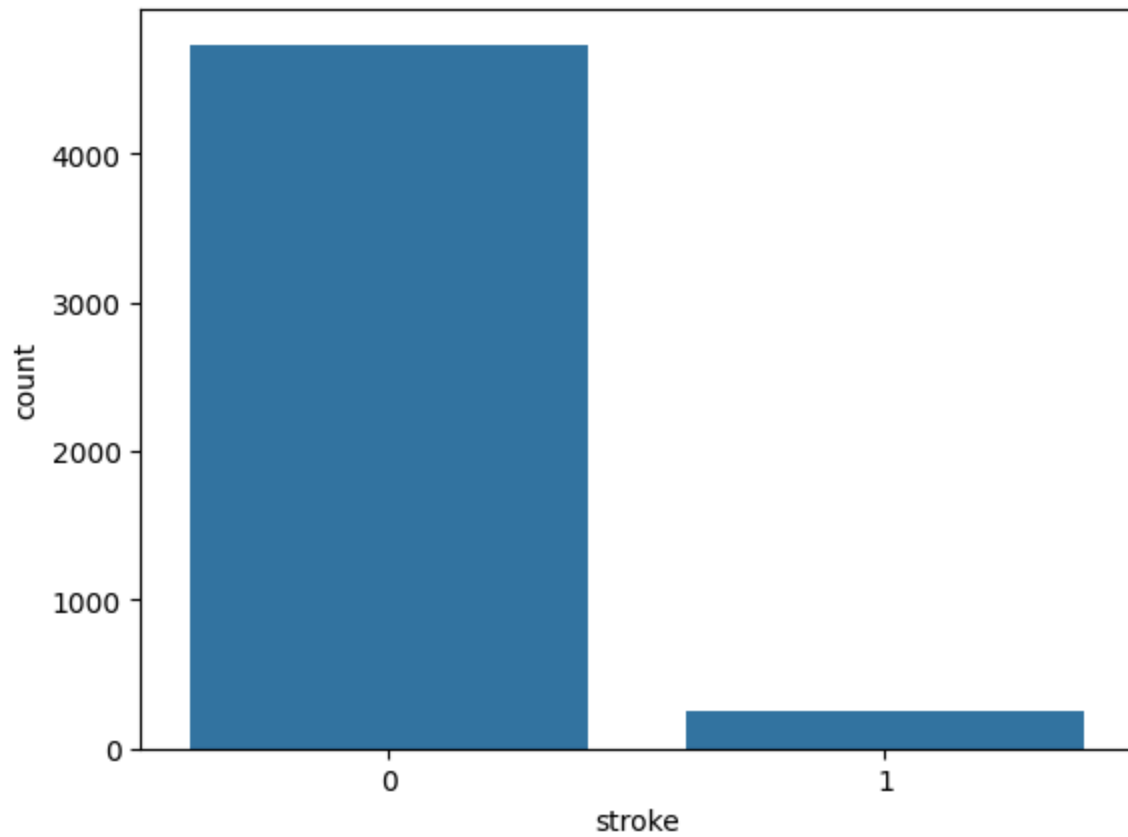
```
Out[ ]: <Axes: xlabel='gender', ylabel='count'>
```



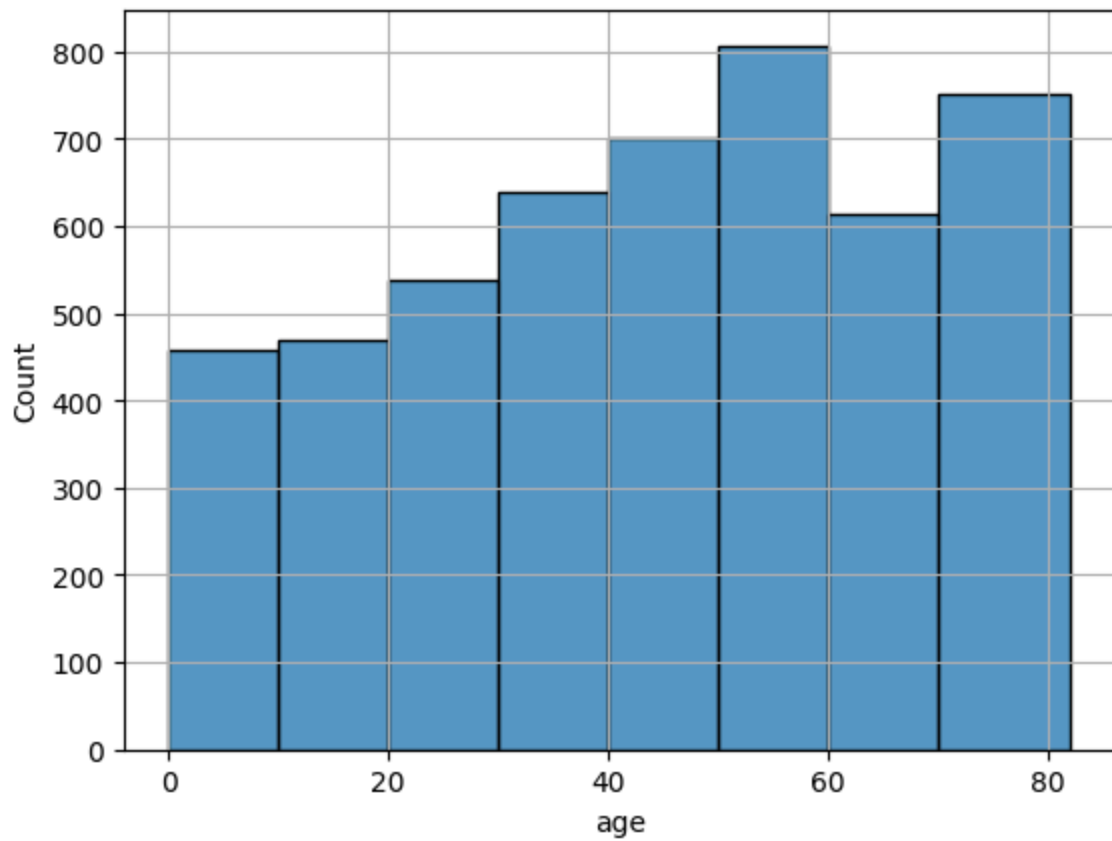
```
In [ ]: print(df['stroke'].value_counts(normalize = True))
sns.countplot(x = df['stroke'])
```

```
stroke
0    0.950211
1    0.049789
Name: proportion, dtype: float64
```

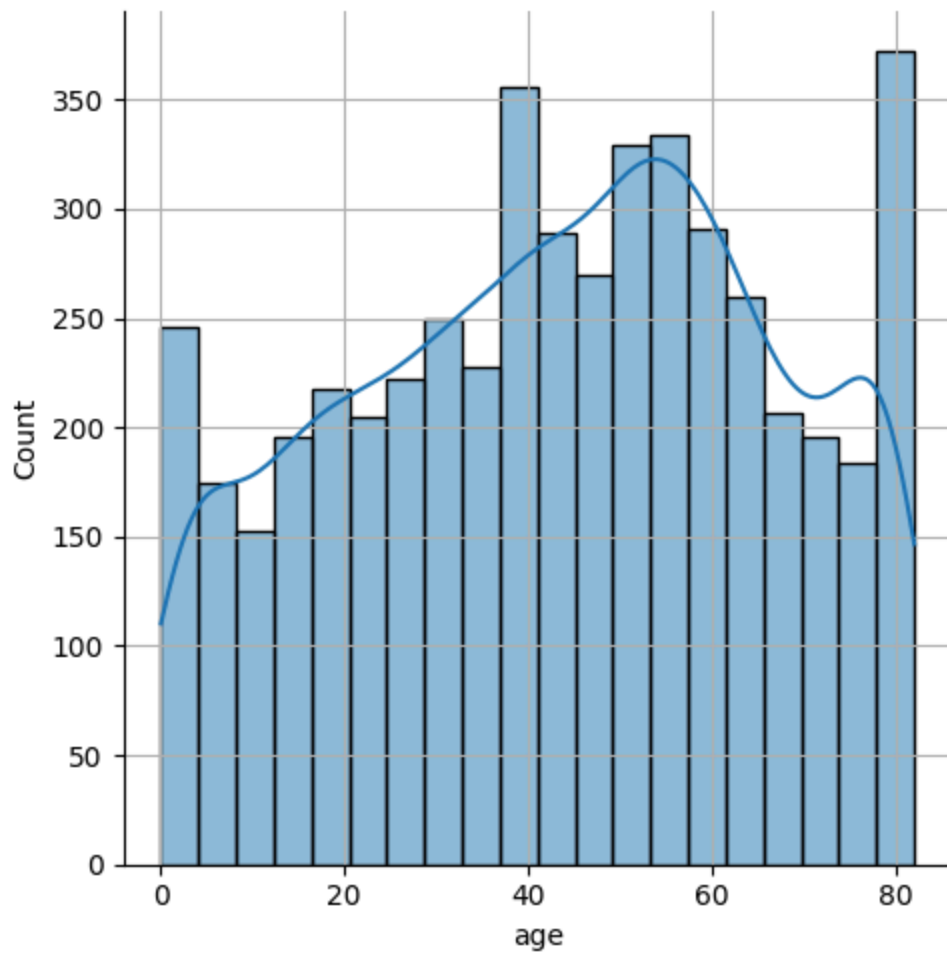
```
Out[ ]: <Axes: xlabel='stroke', ylabel='count'>
```



```
In [ ]: sns.histplot(df['age'], bins= [0,10,20,30,40,50,60,70,82])  
plt.grid()
```



```
In [ ]: sns.displot(df['age'],kde = True)
plt.grid()
```

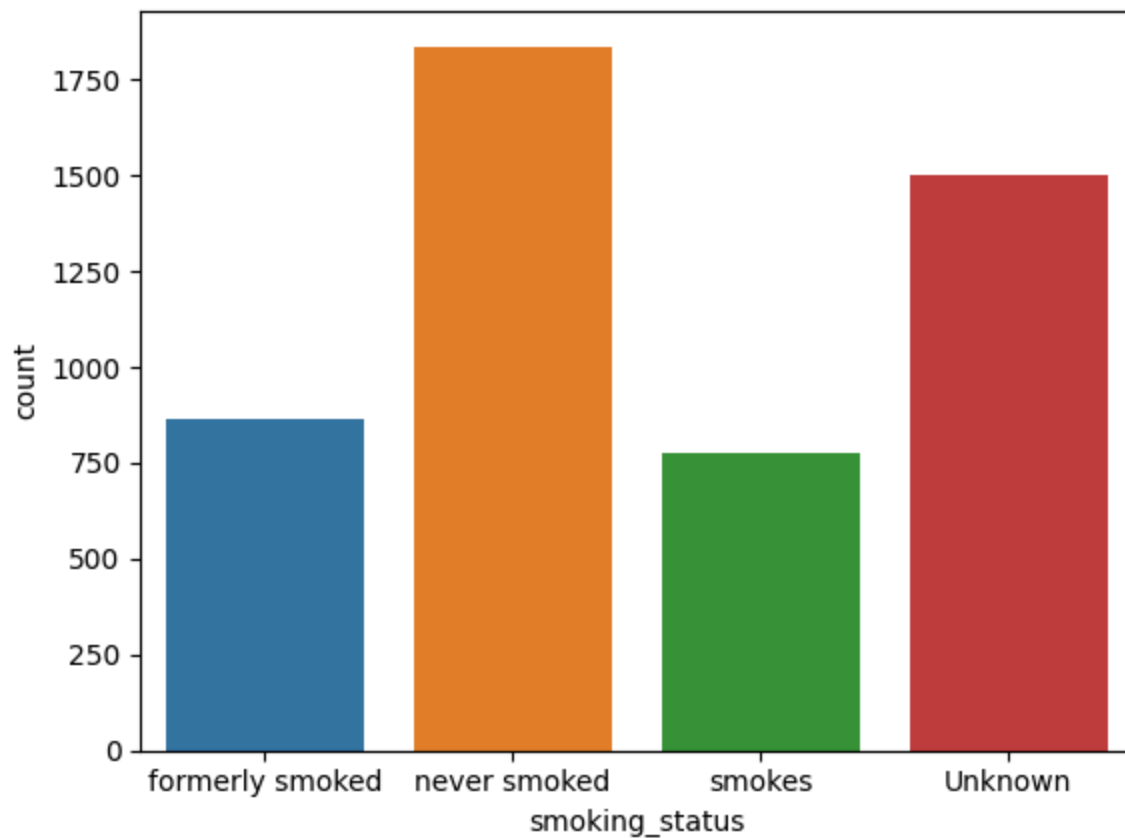


```
In [ ]: print(df['age'].value_counts())
# sns.countplot(df['age'])
```

```
age
78.00    102
57.00     92
54.00     85
51.00     84
79.00     84
...
1.16       3
0.48       3
0.40       2
0.08       2
0.16       1
Name: count, Length: 104, dtype: int64
```

```
In [ ]: print(df['smoking_status'].value_counts(normalize = True))
sns.countplot(x = df['smoking_status'], hue = df['smoking_status'])
plt.show()
```

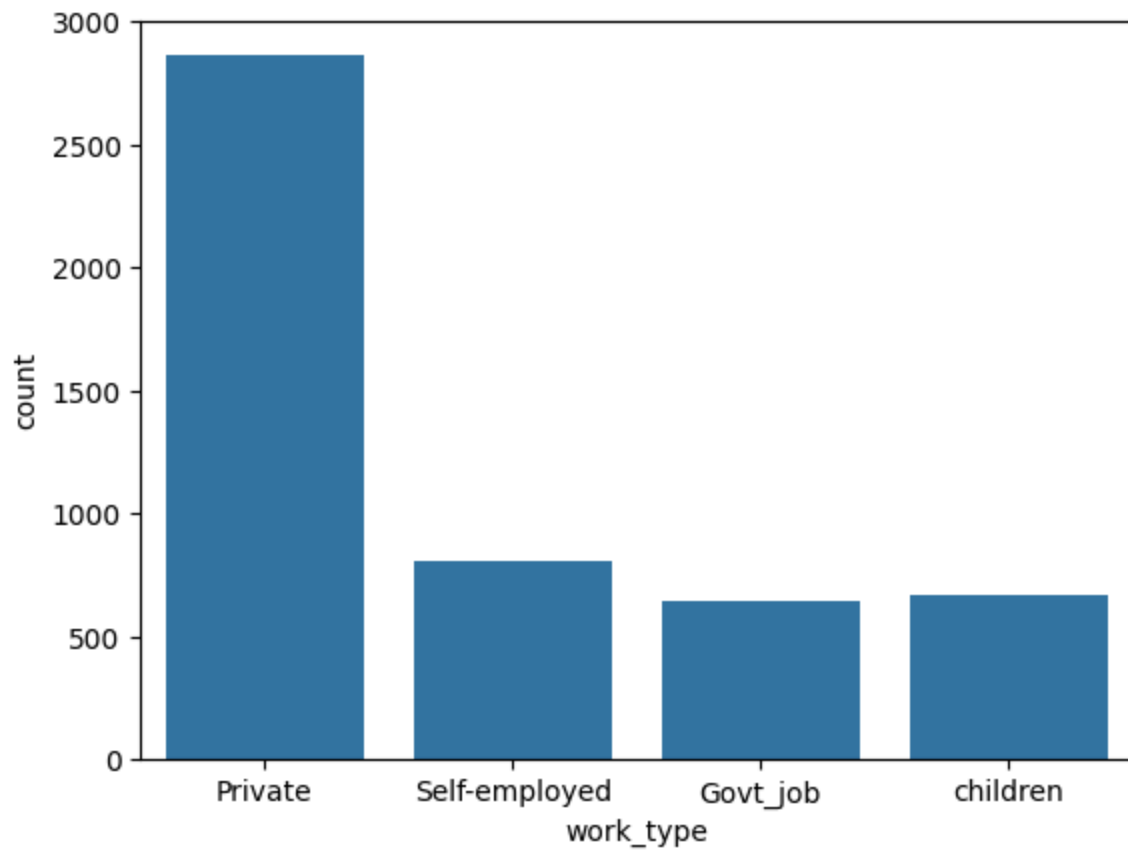
```
smoking_status
never smoked    0.369002
Unknown         0.301144
formerly smoked  0.174061
smokes          0.155792
Name: proportion, dtype: float64
```



```
In [ ]: print(df['work_type'].value_counts())
sns.countplot(x = df['work_type'])
```

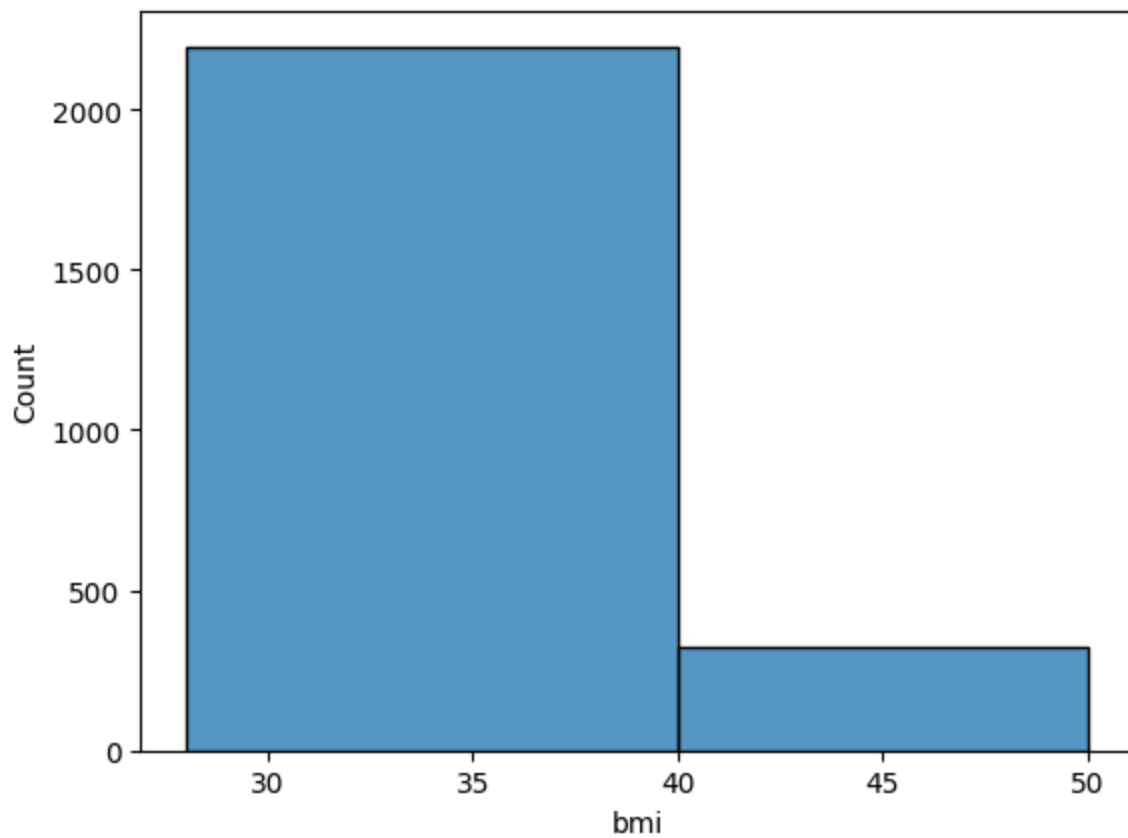
```
work_type
Private      2860
Self-employed  804
children     673
Govt_job     644
Name: count, dtype: int64
```

```
Out[ ]: <Axes: xlabel='work_type', ylabel='count'>
```



```
In [ ]: sns.histplot(df[df['bmi']>28]['bmi'], bins = [28,40,50])
```

```
Out[ ]: <Axes: xlabel='bmi', ylabel='Count'>
```



```
In [ ]: df[df['bmi']>28].shape
```

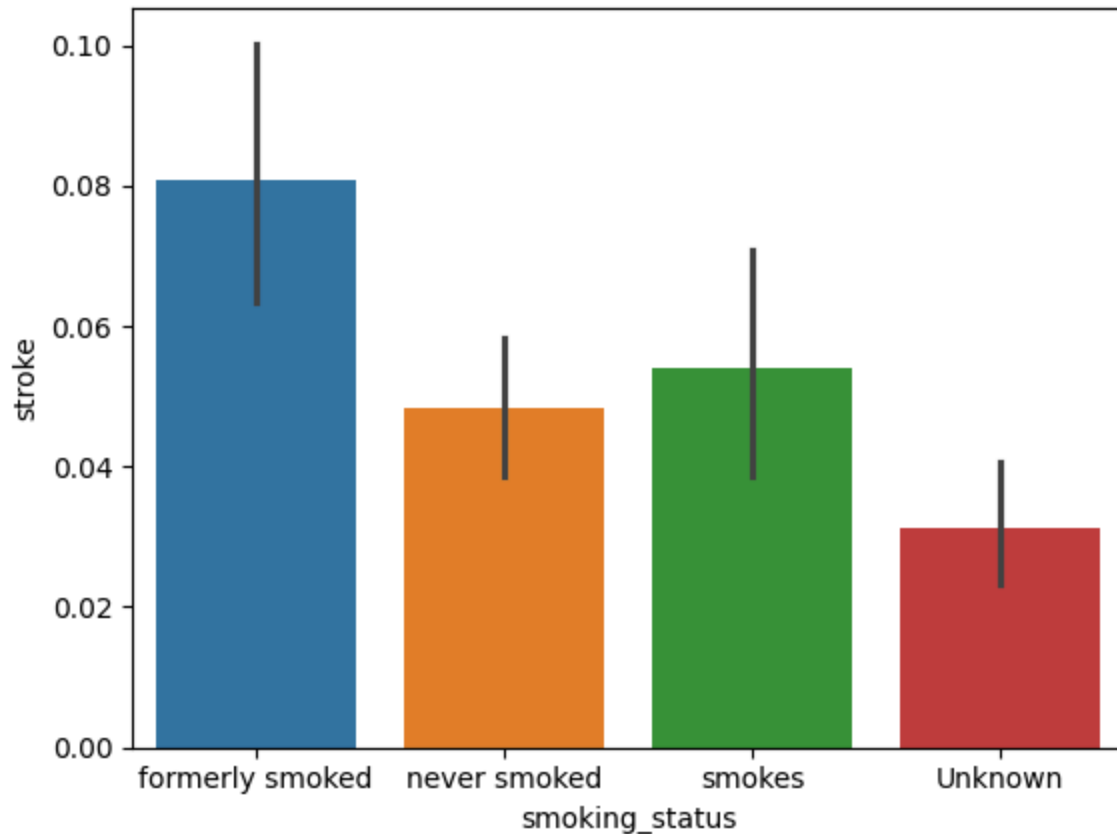
```
Out[ ]: (2518, 11)
```

```
In [ ]: df[df['bmi']>25]['bmi'].count()
```

```
Out[ ]: 3388
```

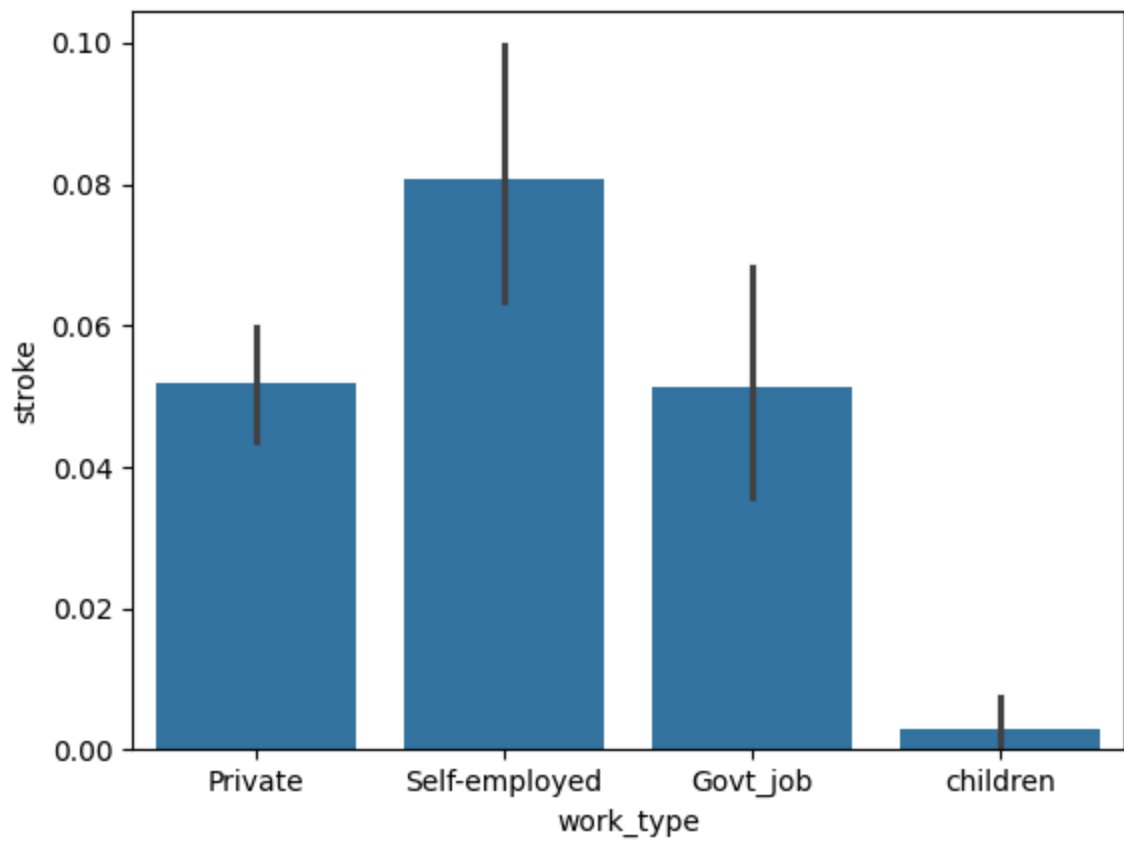
```
In [ ]: sns.barplot(y = df['stroke'], x = df['smoking_status'], hue =df['smoking_status'])
```

```
Out[ ]: <Axes: xlabel='smoking_status', ylabel='stroke'>
```



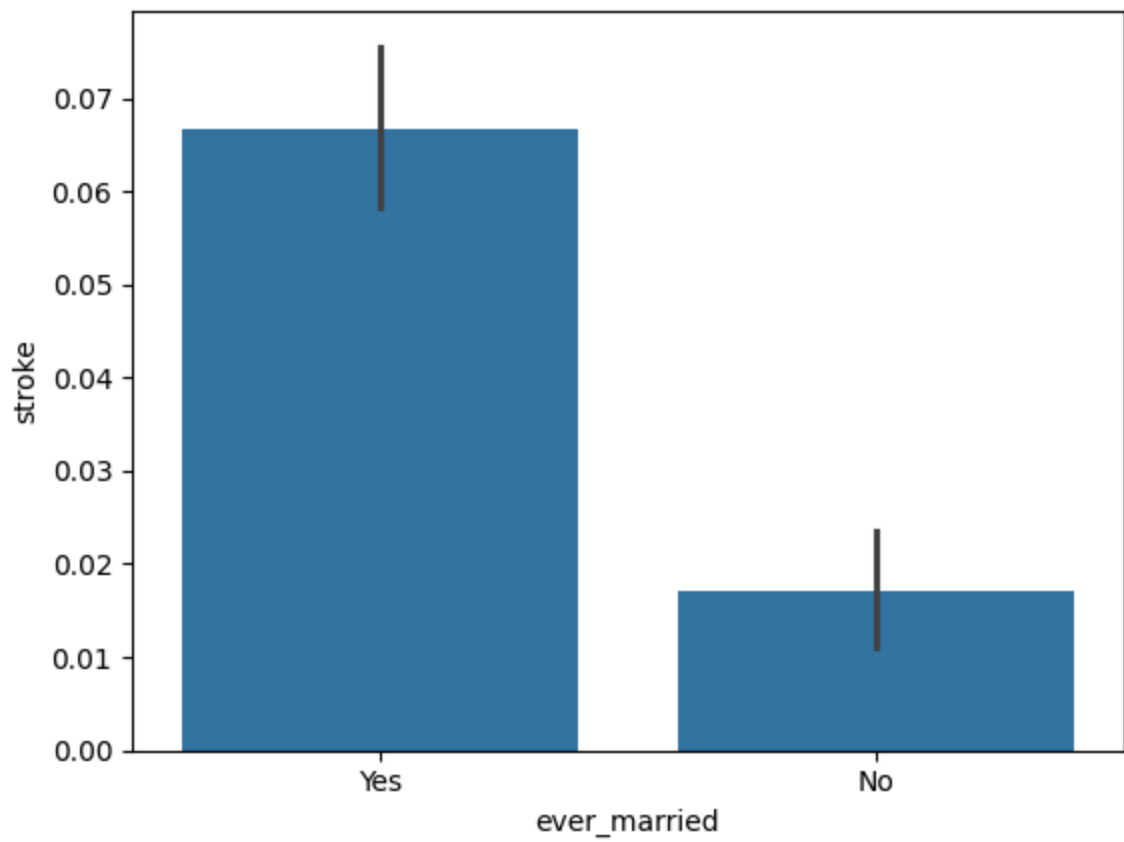
```
In [ ]: sns.barplot(y = df['stroke'], x = df['work_type'])
```

```
Out[ ]: <Axes: xlabel='work_type', ylabel='stroke'>
```



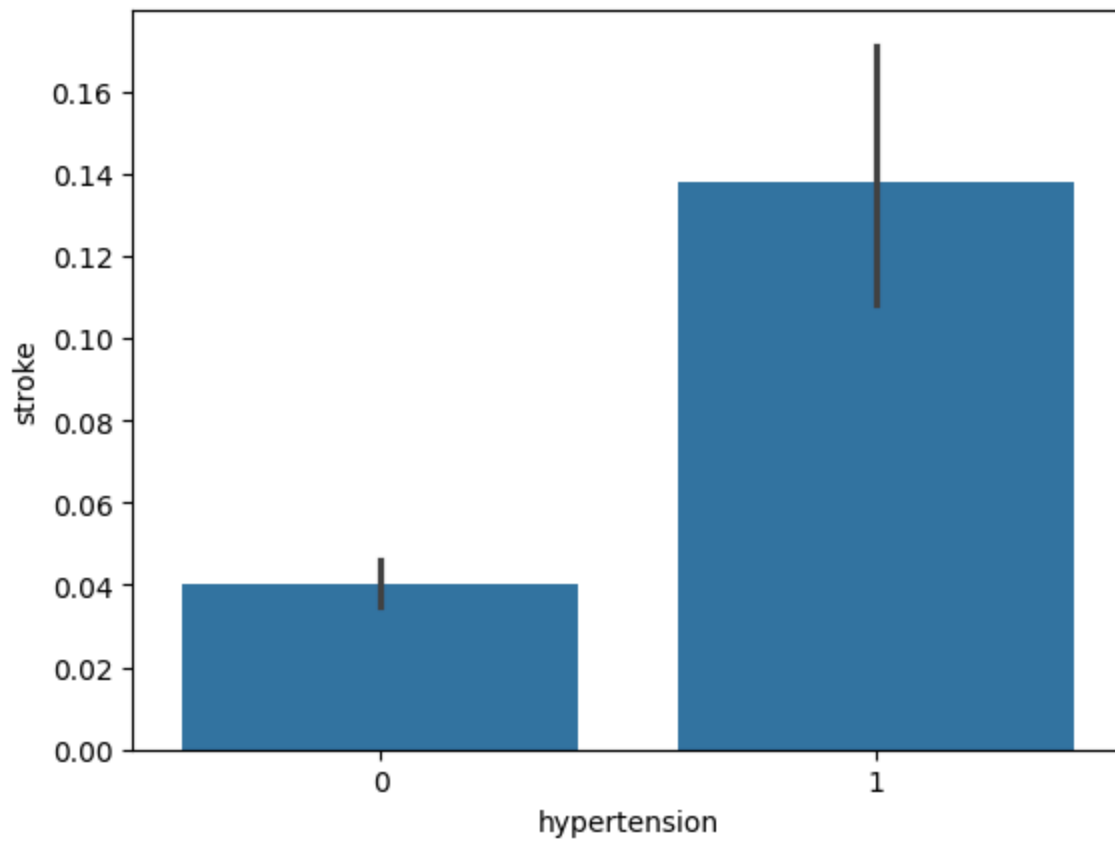
```
In [ ]: sns.barplot(x = df['ever_married'], y = df['stroke'])
```

```
Out[ ]: <Axes: xlabel='ever_married', ylabel='stroke'>
```



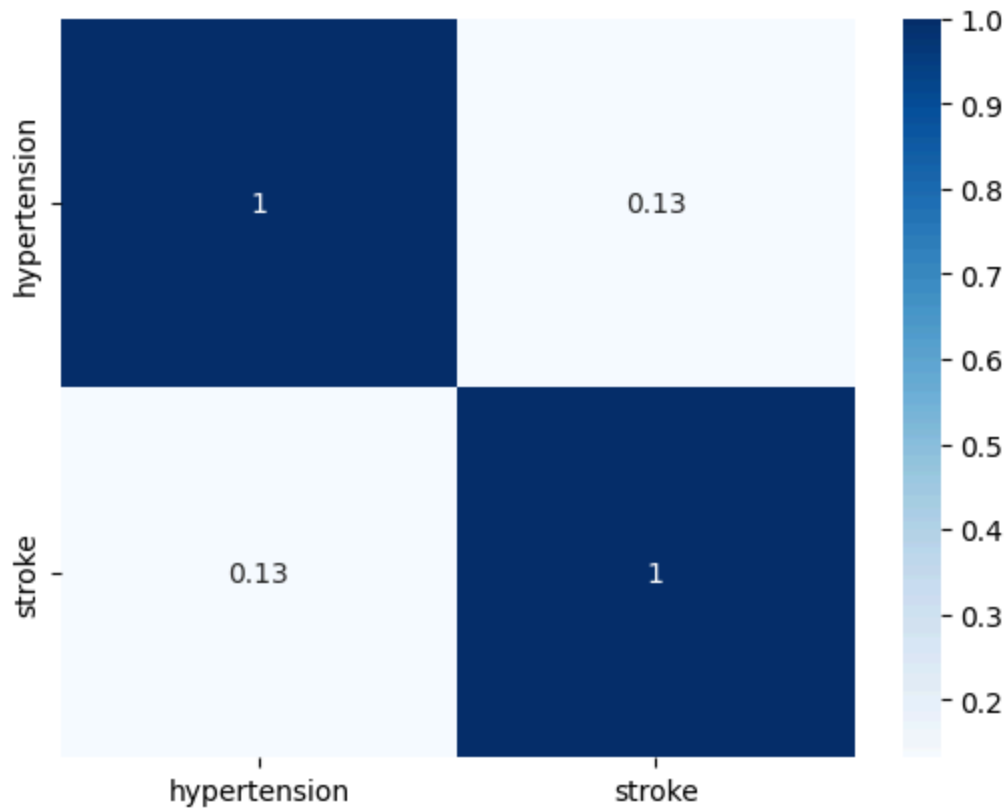
```
In [ ]: sns.barplot(x = df['hypertension'], y = df['stroke'])
```

```
Out[ ]: <Axes: xlabel='hypertension', ylabel='stroke'>
```



```
In [ ]: sns.heatmap(df[['hypertension', 'stroke']].corr(), annot = True, cmap = 'Blues')
```

```
Out[ ]: <Axes: >
```



```
In [ ]: y = x*(1/7.7) # equation of if we want to predict the stroke only behalf of hyperten
y = x*(1/4)
```

```
In [ ]: 5*(1/7.7)
```

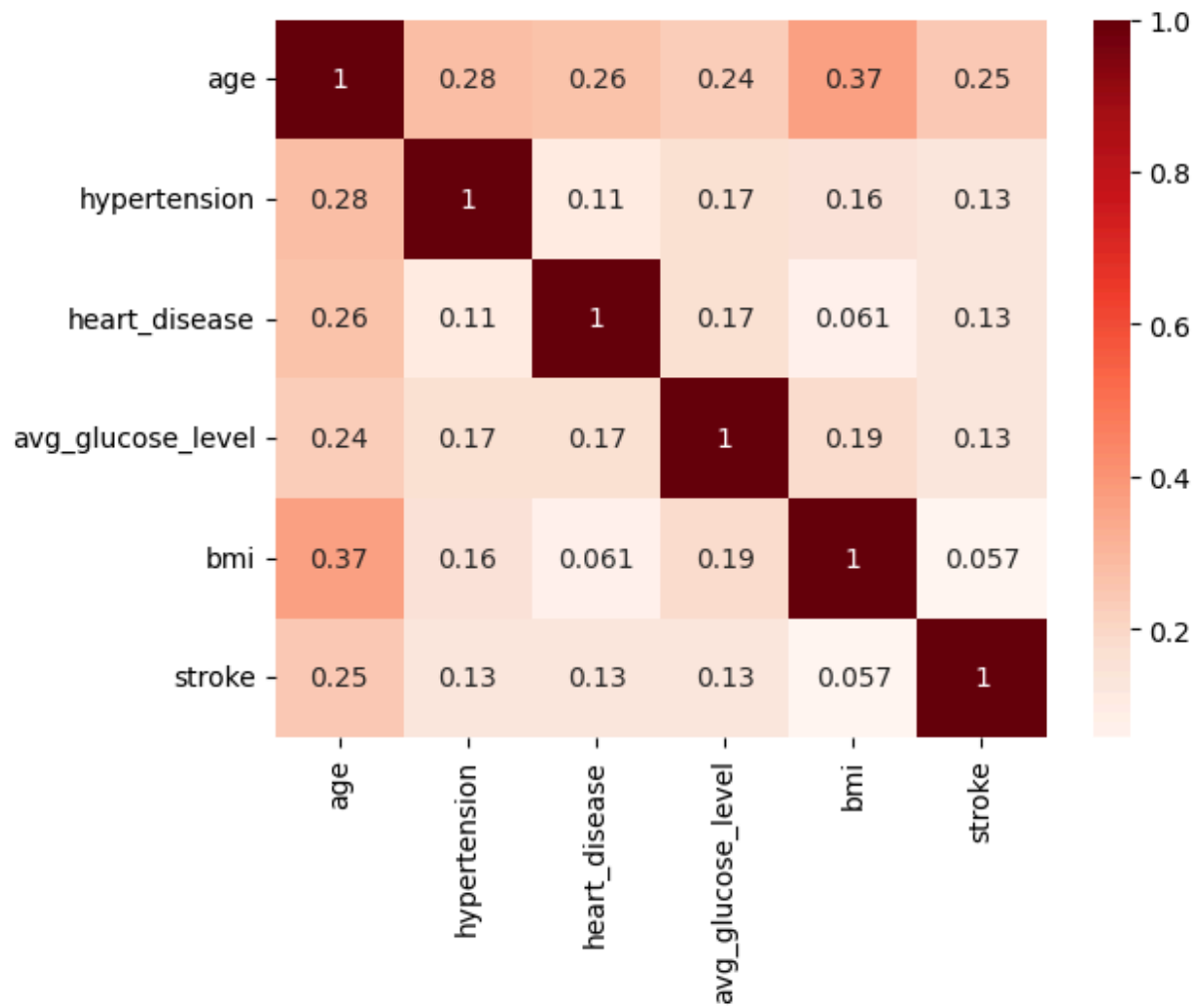
```
Out[ ]: 0.6493506493506493
```

```
In [ ]: df1 = pd.read_clipboard()
df1
```

Out[]:		gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	a
	0	Male	67	0	1	Yes	Private	Urban	
	1	Male	80	0	1	Yes	Private	Rural	
	2	Female	49	0	0	Yes	Private	Urban	
	3	Female	79	1	0	Yes	Self-employed	Rural	
	4	Male	81	0	0	Yes	Private	Urban	
	5	Male	74	1	1	Yes	Private	Rural	
	6	Female	69	0	0	No	Private	Urban	
	7	Female	78	0	0	Yes	Private	Urban	
	8	Female	81	1	0	Yes	Private	Rural	
	9	Female	61	0	1	Yes	Govt_job	Rural	
	10	Female	54	0	0	Yes	Private	Urban	
	11	Female	79	0	1	Yes	Private	Urban	
	12	Female	50	1	0	Yes	Self-employed	Rural	
	13	Male	64	0	1	Yes	Private	Urban	
	14	Male	75	1	0	Yes	Private	Urban	
	15	Female	60	0	0	No	Private	Urban	

```
In [ ]: sns.heatmap(df.corr(numeric_only = True), annot = True, cmap = 'Reds')
```

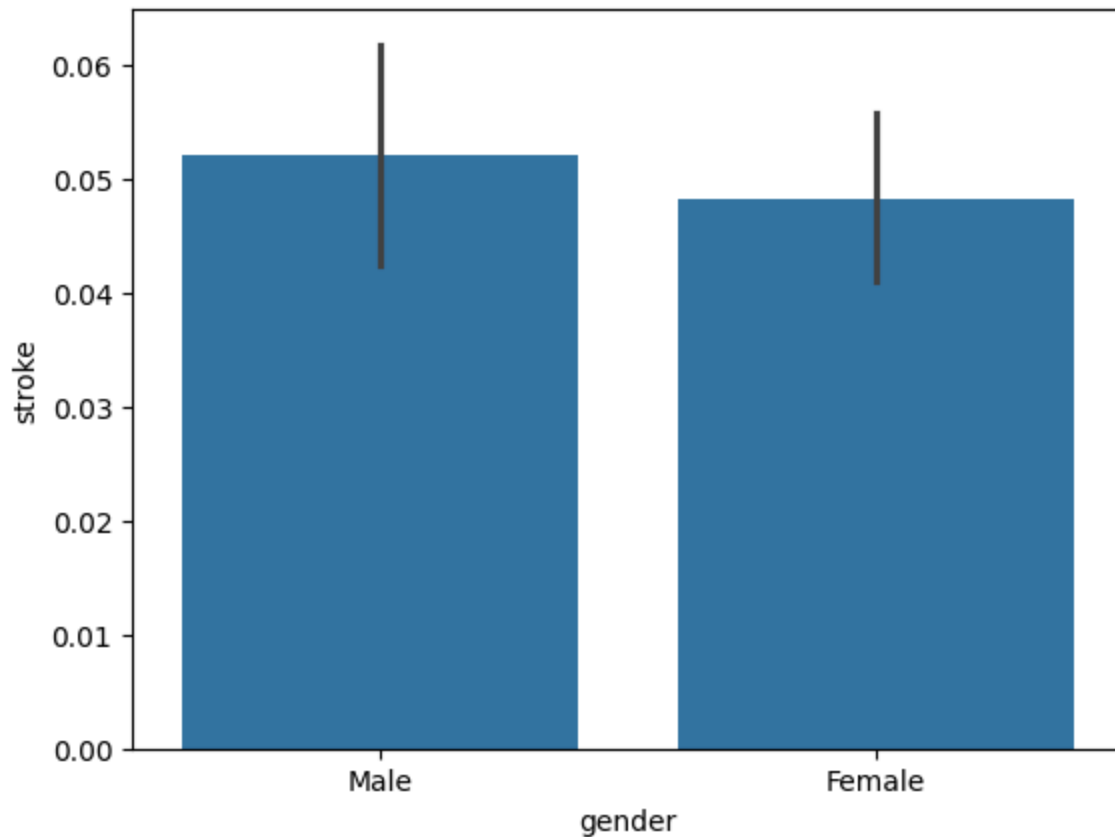
```
Out[ ]: <Axes: >
```



```
In [ ]: df.info()
```

```
In [ ]: sns.barplot(x = df['gender'], y = df['stroke'])
```

```
Out[ ]: <Axes: xlabel='gender', ylabel='stroke'>
```



```
In [ ]: df[df['stroke']==1]['gender'].value_counts()
```

```
Out[ ]: gender
Female    140
Male      108
Name: count, dtype: int64
```

```
In [ ]: df['gender'].value_counts()
```

```
Out[ ]: gender
Female    2907
Male      2074
Name: count, dtype: int64
```

```
In [ ]: 108/2074
(140/100)/(2907/100)
```

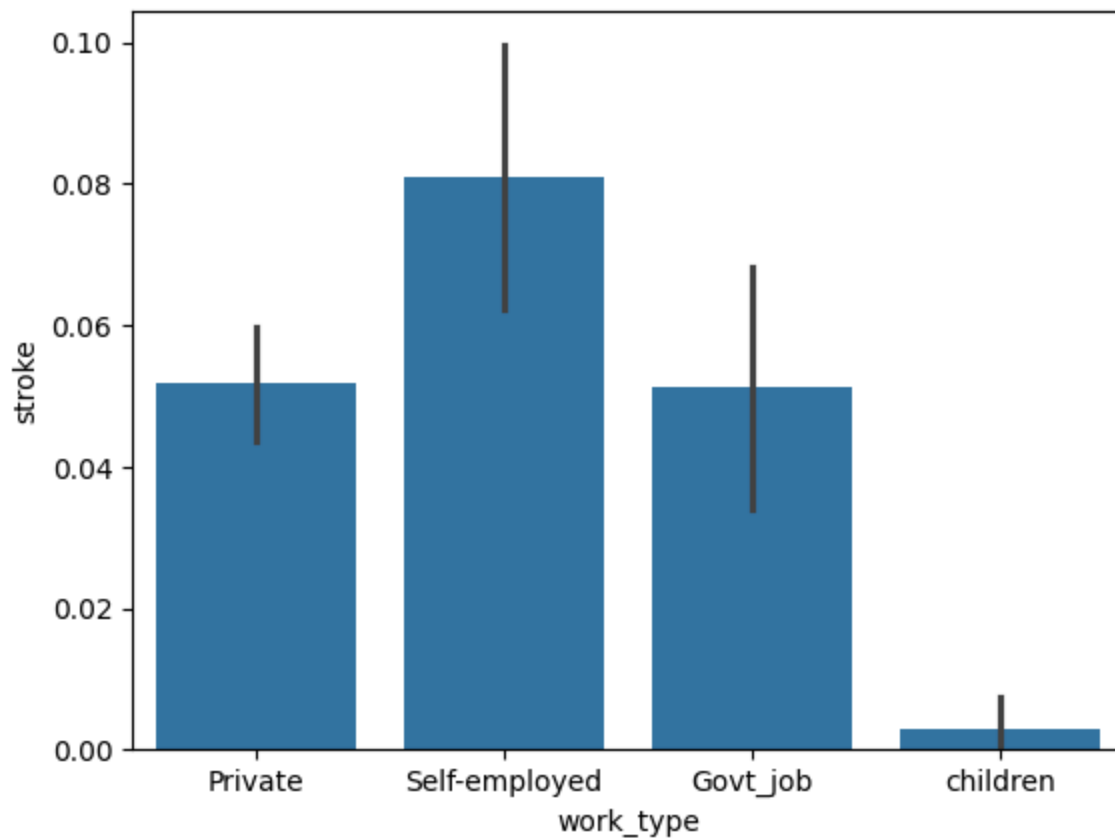
```
Out[ ]: 0.04815961472308221
```

```
In [ ]: ## Tell the number of persons having stroke in private worktype
df[df['work_type'] == 'Private']['stroke'].value_counts()
```

```
Out[ ]: stroke
0      2712
1       148
Name: count, dtype: int64
```

```
In [ ]: sns.barplot(x = df['work_type'], y = df['stroke'])
```

Out[]: <Axes: xlabel='work_type', ylabel='stroke'>



```
In [ ]: ## Tell the number of females having stroke whoes work type is private
df[(df['work_type'] == 'Private') & (df['gender'] == 'Female')]['stroke'].value_co
```

```
Out[ ]: stroke
0      0.955986
1      0.044014
Name: proportion, dtype: float64
```

```
In [ ]: ## Tell the number of females having stroke whoes work type is private
df[(df['work_type'] == 'Private') & (df['gender'] == 'Male')]['stroke'].value_coun
```

```
Out[ ]: stroke
0      0.936851
1      0.063149
Name: proportion, dtype: float64
```

```
In [ ]: df['smoking_status'].value_counts(normalize = True) # show the percentage of data u
```

```
Out[ ]: smoking_status
never smoked      0.369002
Unknown           0.301144
formerly smoked   0.174061
smokes            0.155792
Name: proportion, dtype: float64
```

```
In [ ]: df.groupby('work_type')['hypertension'].mean()
```

```
Out[ ]: work_type
Govt_job      0.108696
Private       0.095105
Self-employed 0.170398
children      0.000000
Name: hypertension, dtype: float64
```

```
In [ ]: df.groupby('Residence_type')['hypertension'].mean()
```

```
Out[ ]: Residence_type
Rural    0.097591
Urban    0.094787
Name: hypertension, dtype: float64
```

```
In [ ]: df.groupby('Residence_type')['stroke'].mean()
```

```
Out[ ]: Residence_type
Rural    0.046141
Urban    0.053318
Name: stroke, dtype: float64
```

```
In [ ]: data = {'Hypertens':[0,0,1,1,1,0,1,1,0,1,1,1,0,1,0],
               'work_type':['p', 'p', 'g','g','p','p','g','p','p','p','p','g','p','g','p']}
d = pd.DataFrame(data)
d.describe()
d.groupby('work_type')['Hypertens'].mean()
```

```
Out[ ]: work_type
g      1.0
p      0.4
Name: Hypertens, dtype: float64
```

```
In [ ]: d
```

```
In [ ]: len(data['Hypertens'])
```

```
In [ ]: df.groupby(['work_type','hypertension'])['stroke'].mean()
```

```
Out[ ]: work_type      hypertension
Govt_job      0              0.043554
              1              0.114286
Private       0              0.044436
              1              0.121324
Self-employed 0              0.059970
              1              0.182482
children      0              0.002972
Name: stroke, dtype: float64
```

```
In [ ]: df.groupby(['work_type','ever_married'])['stroke'].mean()
```

```
Out[ ]: work_type      ever_married
Govt_job      No          0.052632
              Yes          0.050943
Private       No          0.017370
              Yes          0.065239
Self-employed No          0.064815
              Yes          0.083333
children      No          0.002972
Name: stroke, dtype: float64
```

```
In [ ]: df.sort_values(by=['bmi', 'avg_glucose_level'], ascending=[True, True]).head(20)
```

```
Out[ ]:
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
4223	Female	4.00	0	0	No	children	Urbar
1776	Female	67.00	0	0	Yes	Private	Urbar
2609	Female	8.00	0	0	No	children	Rura
3193	Male	0.56	0	0	No	children	Urbar
3907	Female	3.00	0	0	No	children	Rura
1487	Female	0.08	0	0	No	children	Urbar
3148	Male	0.24	0	0	No	children	Urbar
3939	Female	3.00	0	0	No	children	Rura
2052	Male	2.00	0	0	No	children	Rura
2468	Male	61.00	0	0	Yes	Private	Urbar
2682	Male	10.00	0	0	No	children	Urbar
1474	Male	0.40	0	0	No	children	Urbar
1081	Female	11.00	0	0	No	children	Rura
3277	Female	11.00	0	0	No	children	Urbar
615	Female	1.80	0	0	No	children	Urbar
3083	Female	9.00	0	0	No	children	Urbar
3368	Male	8.00	0	0	No	children	Urbar
1126	Male	10.00	0	0	No	children	Rura
740	Female	5.00	0	0	No	children	Urbar
4478	Male	8.00	0	0	No	children	Rura

```
In [ ]: df['stroke'].value_counts()
```

```
Out[ ]: stroke
0      4733
1       248
Name: count, dtype: int64
```

```
In [ ]: df[df['stroke'] == 1].sort_values(by='age').head(30)
```

Out[]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
4807	Female	1.32	0	0	No	children	Urbar
205	Female	14.00	0	0	No	children	Rura
148	Female	32.00	0	0	Yes	Private	Rura
98	Female	38.00	0	0	No	Self-employed	Urbar
4802	Female	38.00	0	0	Yes	Private	Rura
172	Female	39.00	0	0	Yes	Private	Urbar
41	Female	39.00	1	0	Yes	Private	Rura
188	Female	39.00	0	0	Yes	Self-employed	Urbar
25	Male	42.00	0	0	Yes	Private	Rura
64	Male	43.00	0	0	Yes	Self-employed	Urbar
77	Male	45.00	0	0	Yes	Private	Rura
51	Female	45.00	0	0	Yes	Private	Rura
146	Female	46.00	0	0	Yes	Private	Urbar
71	Male	47.00	0	0	Yes	Private	Urbar
28	Male	48.00	0	0	No	Govt_job	Urbar
175	Female	48.00	0	0	Yes	Private	Urbar
33	Female	49.00	0	0	Yes	Private	Urbar
2	Female	49.00	0	0	Yes	Private	Urbar
185	Male	49.00	0	0	No	Private	Rura
130	Female	50.00	0	0	Yes	Private	Rura
88	Female	50.00	1	0	Yes	Private	Rura
12	Female	50.00	1	0	Yes	Self-employed	Rura
198	Female	51.00	1	0	Yes	Private	Urbar
155	Male	51.00	1	0	Yes	Private	Urbar
4794	Female	51.00	0	0	Yes	Private	Urbar

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
173	Female	51.00	0	0	Yes	Govt_job	Rura
17	Female	52.00	1	0	Yes	Self-employed	Urbar
60	Female	52.00	0	0	Yes	Private	Rura
91	Female	53.00	0	0	Yes	Govt_job	Urbar
101	Female	53.00	1	0	Yes	Private	Urbar

```
In [ ]: ## Created a function to change age int/float dtype
## column into categorial column object dtype for more understanding
```

```
def fun(age):
    if age<=13:
        return "kid"
    elif age>13 and age<=22:
        return "teenager"
    elif 22<age<40:
        return 'Adult'
    else:
        return "Mature"
```

```
In [ ]: # Making changes in dataframe age column by using apply function
df["age_cat"]=df["age"].apply(fun)
df
```

```
Out[ ]:
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
0	Male	67.0	0	1	Yes	Private	Urban
1	Male	80.0	0	1	Yes	Private	Rural
2	Female	49.0	0	0	Yes	Private	Urban
3	Female	79.0	1	0	Yes	Self-employed	Rural
4	Male	81.0	0	0	Yes	Private	Urban
...	...	...	...	...	...	...	...
4976	Male	41.0	0	0	No	Private	Rural
4977	Male	40.0	0	0	Yes	Private	Urban
4978	Female	45.0	1	0	Yes	Govt_job	Rural
4979	Male	40.0	0	0	Yes	Private	Rural
4980	Female	80.0	1	0	Yes	Private	Urban

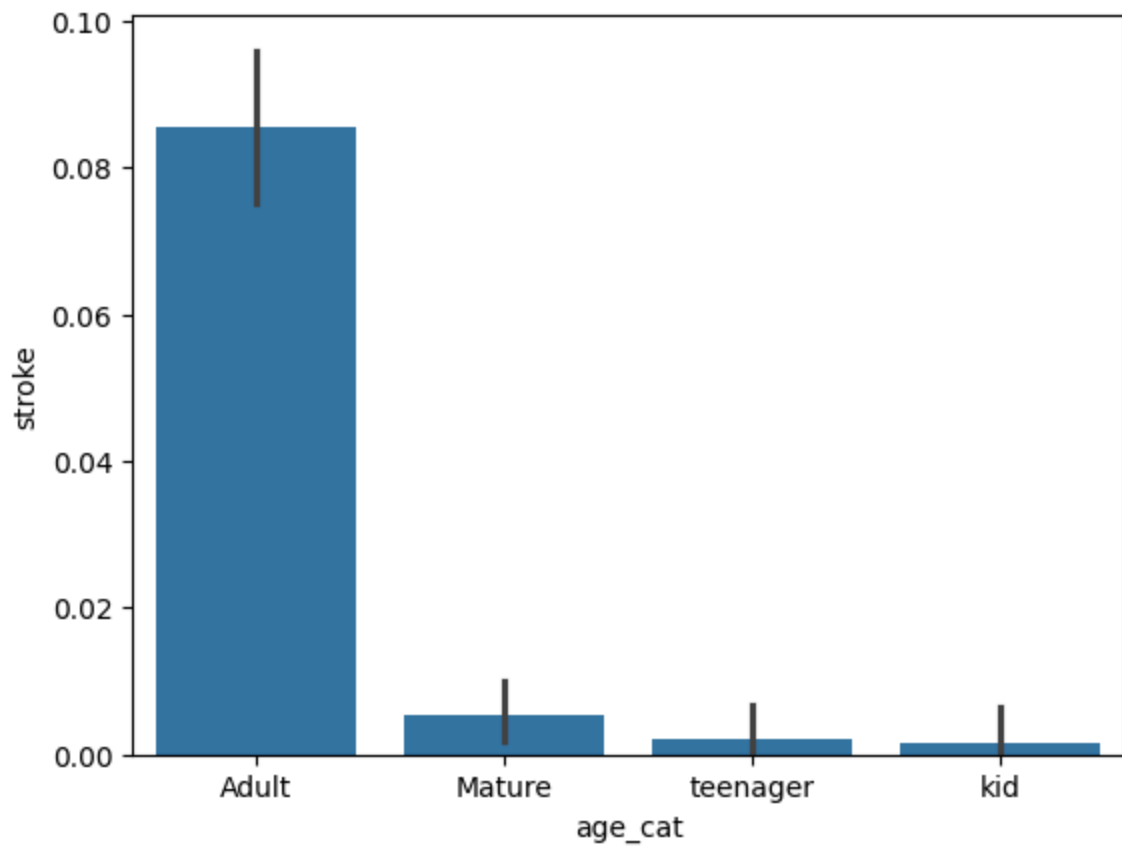
4981 rows × 12 columns

```
In [ ]: df['age_cat'].value_counts()
```

```
Out[ ]: age_cat
Adult      2803
Mature     1099
kid         628
teenager    451
Name: count, dtype: int64
```

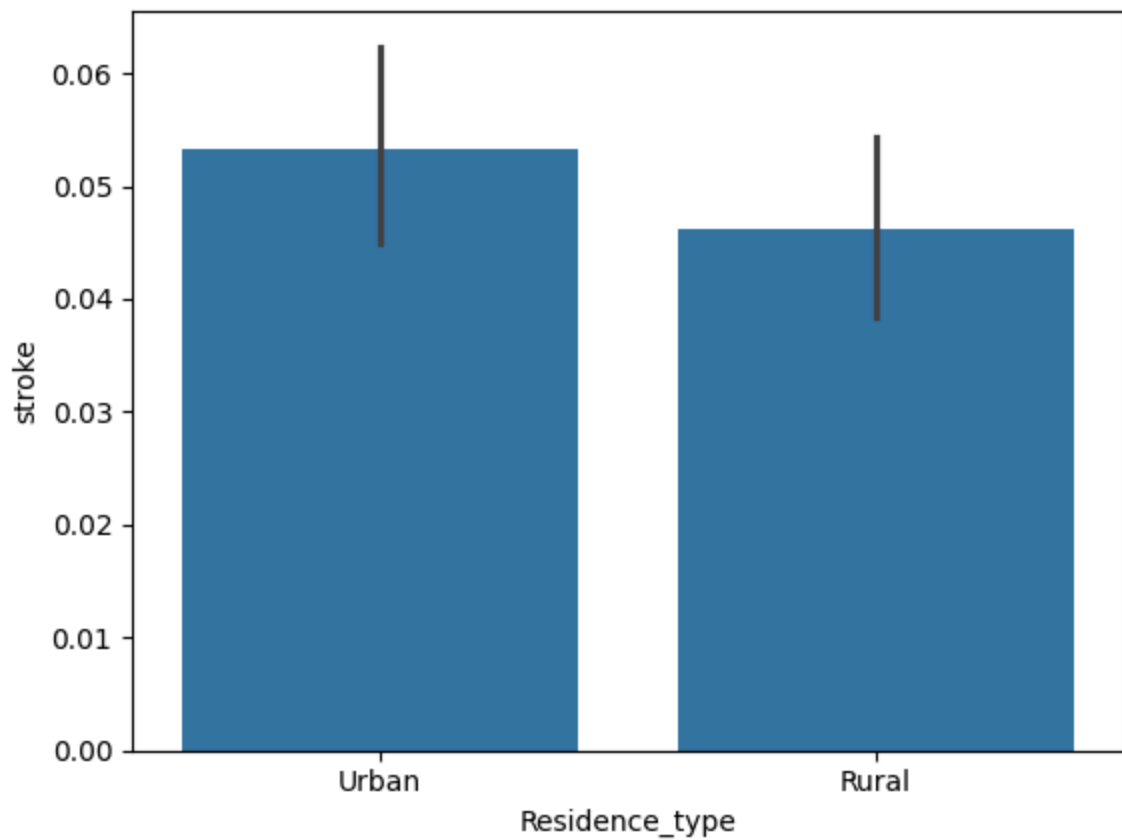
```
In [ ]: sns.barplot(x = df['age_cat'], y = df['stroke'])
```

```
Out[ ]: <Axes: xlabel='age_cat', ylabel='stroke'>
```



```
In [ ]: sns.barplot(x = df['Residence_type'], y = df['stroke'])
```

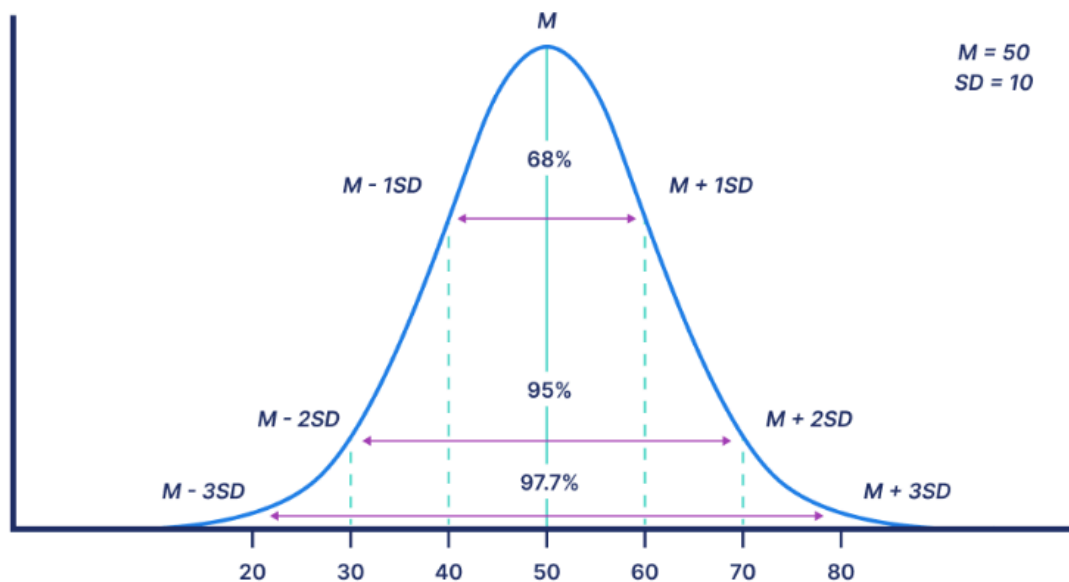
```
Out[ ]: <Axes: xlabel='Residence_type', ylabel='stroke'>
```

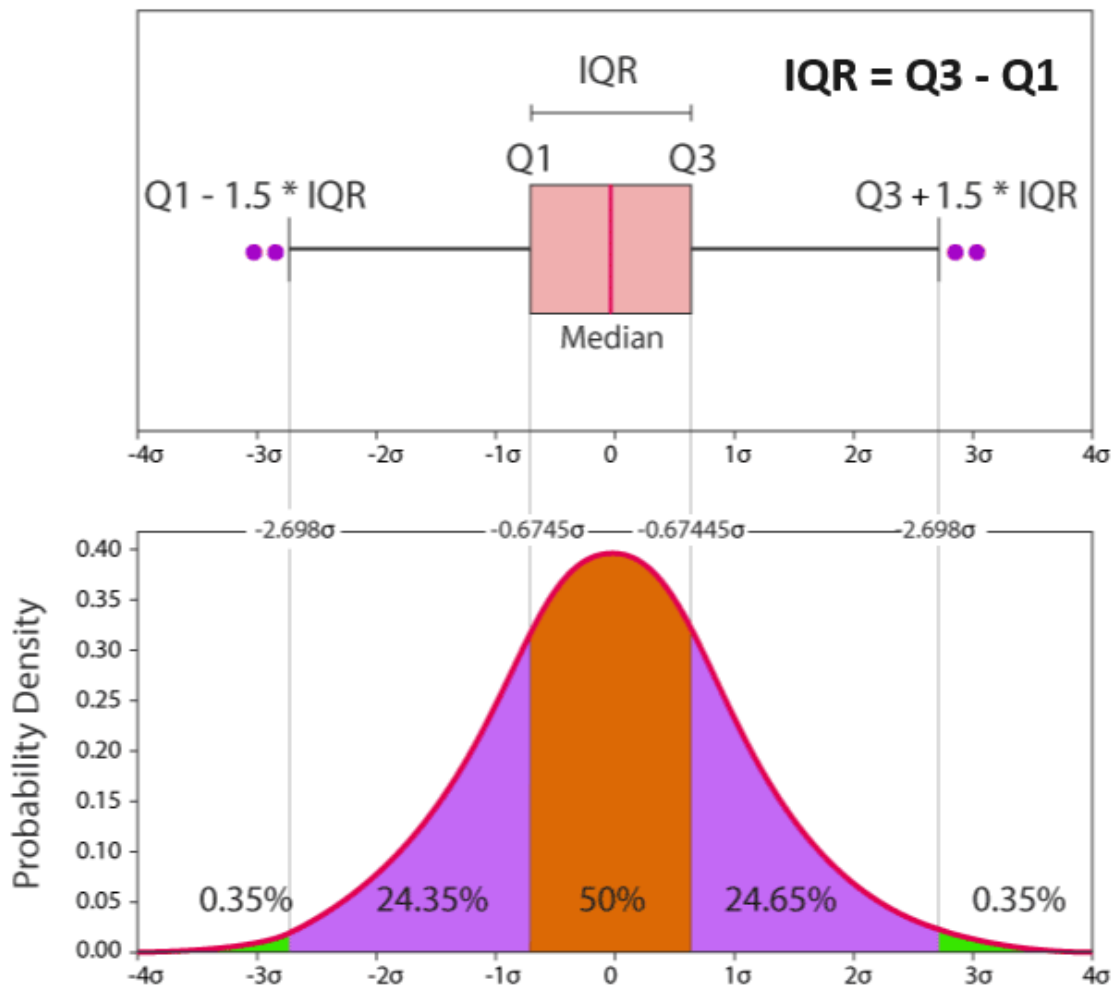


```
In [ ]: df[df['stroke'] == 1]['Residence_type'].value_counts(normalize = True)
```

```
Out[ ]: Residence_type  
Urban    0.544355  
Rural    0.455645  
Name: proportion, dtype: float64
```

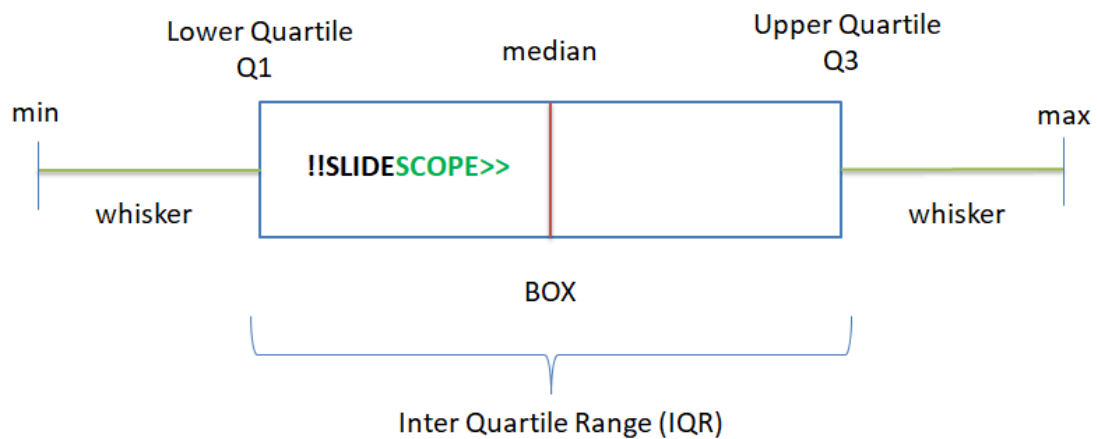
Standard deviations in a normal distribution





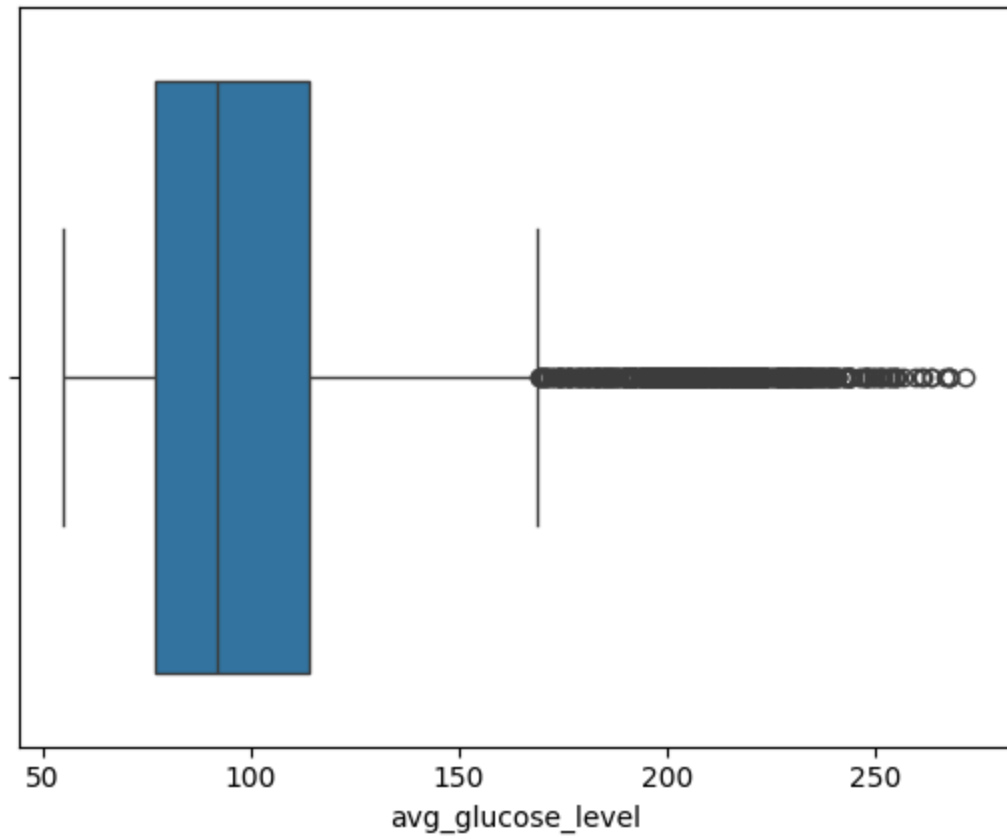
Boxplot on a normal distribution

What is Box Plot ?



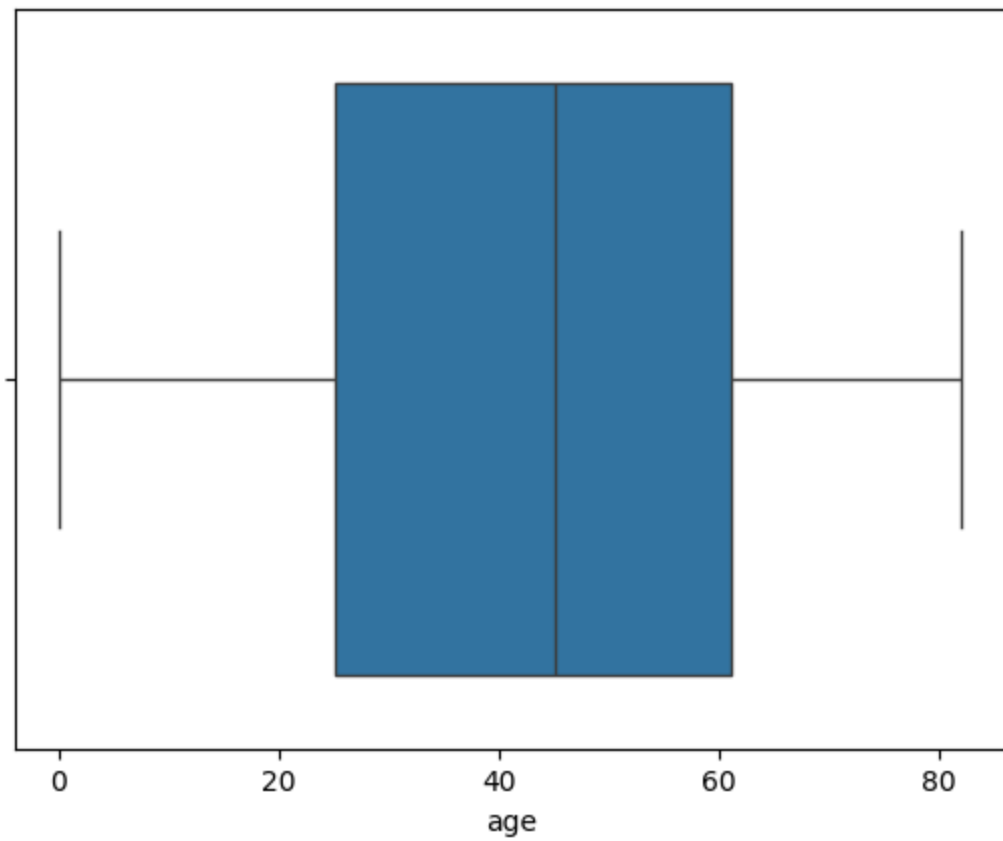
```
In [ ]: sns.boxplot(x="avg_glucose_level", data=df)
```

```
Out[ ]: <Axes: xlabel='avg_glucose_level'>
```



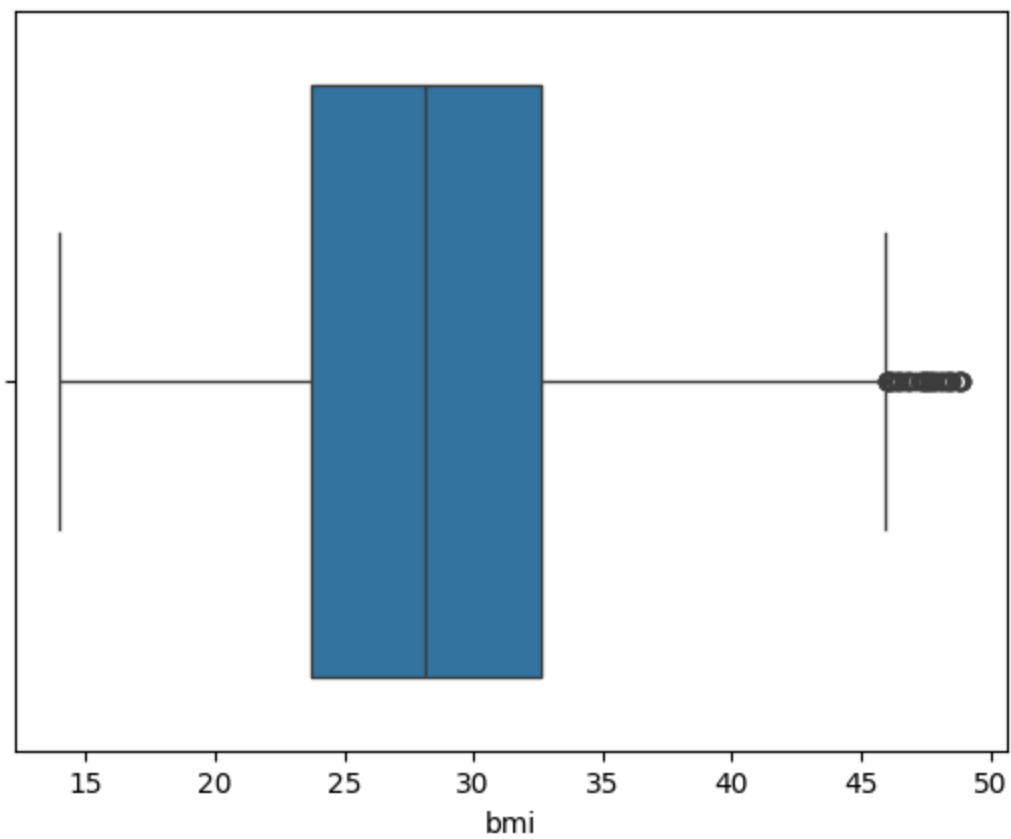
```
In [ ]: sns.boxplot(x="age",data=df)
```

```
Out[ ]: <Axes: xlabel='age'>
```



```
In [ ]: sns.boxplot(x="bmi",data=df)
```

```
Out[ ]: <Axes: xlabel='bmi'>
```



```
In [ ]: df['bmi'].describe()
```

```
Out[ ]: count    4981.000000
        mean      28.498173
        std       6.790464
        min       14.000000
        25%       23.700000
        50%       28.100000
        75%       32.600000
        max       48.900000
        Name: bmi, dtype: float64
```

```
In [ ]:
```

```
In [ ]: a1 = df['bmi'].sort_values(ascending = True).reset_index(drop = True)
        a1
```

```
Out[ ]: 0      14.0
        1      14.1
        2      14.1
        3      14.1
        4      14.1
        ...
        4976   48.8
        4977   48.8
        4978   48.9
        4979   48.9
        4980   48.9
        Name: bmi, Length: 4981, dtype: float64
```

```
In [ ]: q1 = (25/100)*4981
        print(q1)
        a1[np.ceil(q1)]
```

```
1245.25
```

```
Out[ ]: 23.7
```

```
In [ ]: q3 = (75/100)*4981
        print(q3)
        a1[np.ceil(q3)]
```

```
3735.75
```

```
Out[ ]: 32.6
```

```
In [ ]: # IQR = q3-q1
        # IQR
        IQR = 32.6-23.7
        IQR
```

```
Out[ ]: 8.9000000000000002
```

```
In [ ]: # lower_outlier = q1-(1.5*IQR)
        # upper_outlier = q3+(1.5*IQR)
```

```
lower_outlier = 23.7-(1.5*8.9)
lower_outlier
```

Out[]: 10.349999999999998

```
upper_outlier = 32.6+(1.5*8.9)
upper_outlier
```

Out[]: 45.95

```
a1[a1>45.95].count()
```

Out[]: 43

```
df[df['bmi']<45.95]
```

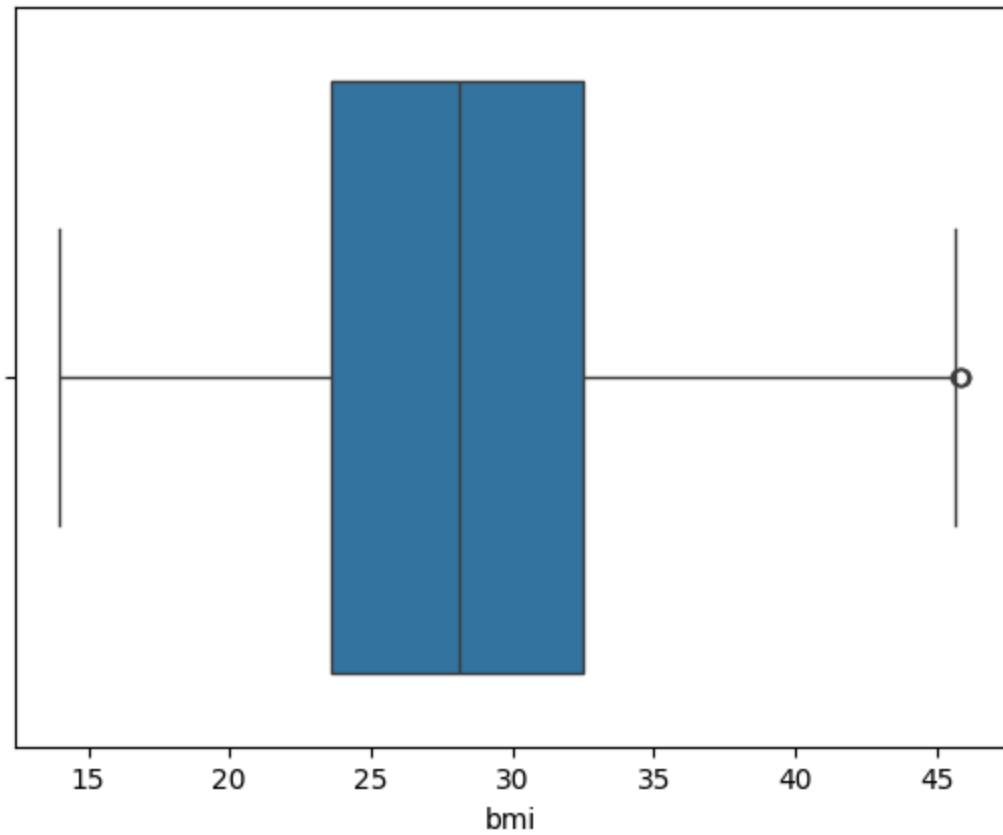
Out[]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
0	Male	67.0	0	1	Yes	Private	Urban
1	Male	80.0	0	1	Yes	Private	Rural
2	Female	49.0	0	0	Yes	Private	Urban
3	Female	79.0	1	0	Yes	Self-employed	Rural
4	Male	81.0	0	0	Yes	Private	Urban
...	...	...	...	...	...	...	...
4976	Male	41.0	0	0	No	Private	Rural
4977	Male	40.0	0	0	Yes	Private	Urban
4978	Female	45.0	1	0	Yes	Govt_job	Rural
4979	Male	40.0	0	0	Yes	Private	Rural
4980	Female	80.0	1	0	Yes	Private	Urban

4938 rows × 12 columns

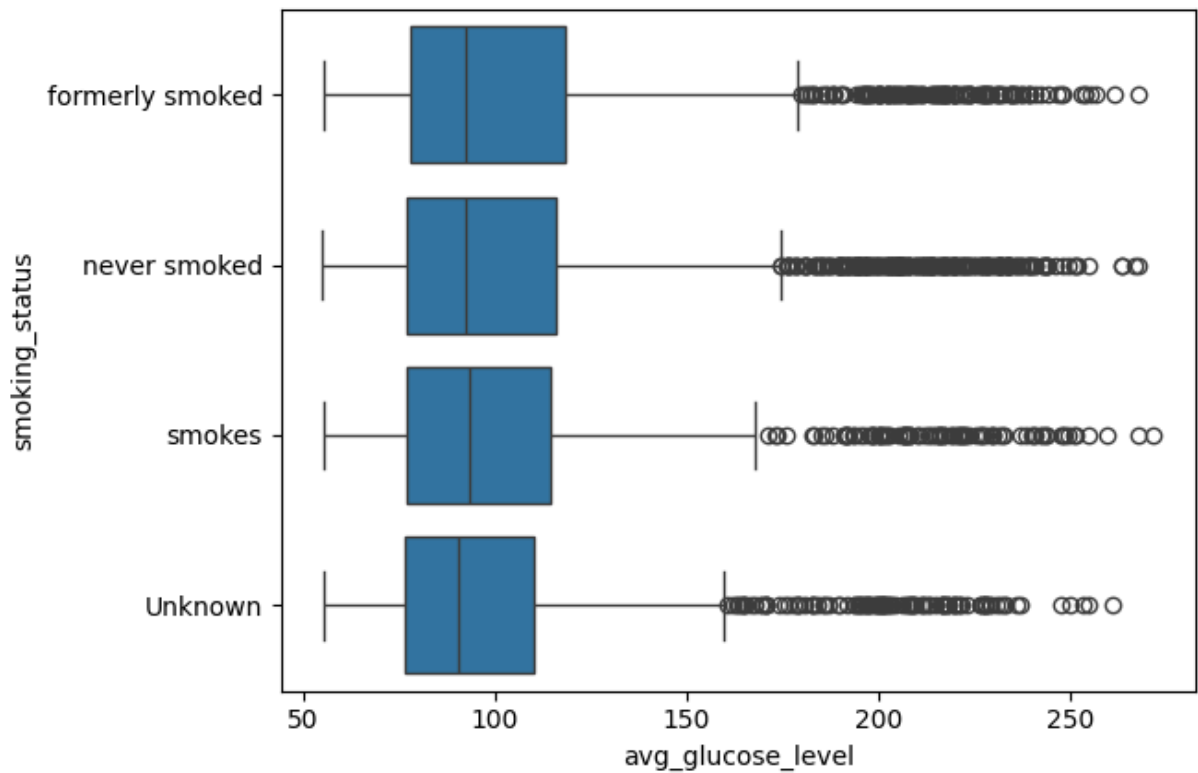
```
sns.boxplot(x='bmi',data=df[df['bmi']<45.95])
```

Out[]: <Axes: xlabel='bmi'>



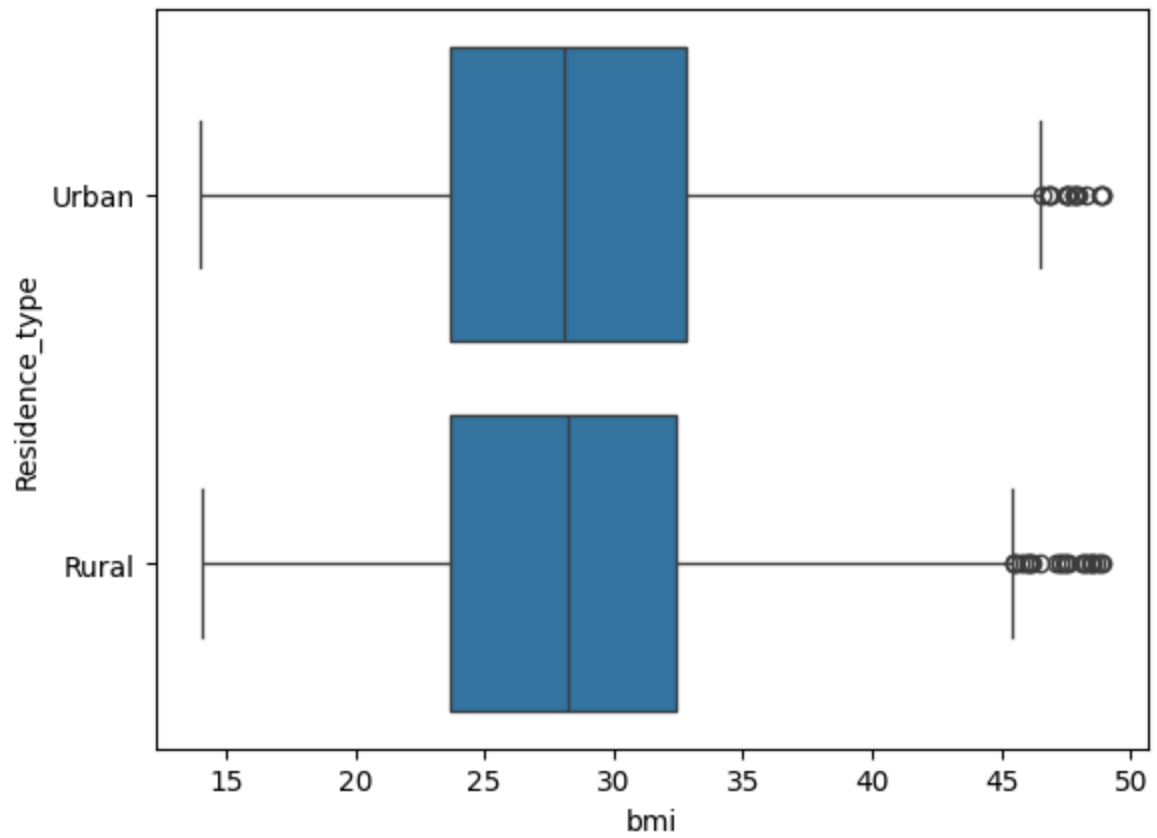
```
In [ ]: sns.boxplot(x="avg_glucose_level", y = 'smoking_status', data=df)
```

```
Out[ ]: <Axes: xlabel='avg_glucose_level', ylabel='smoking_status'>
```



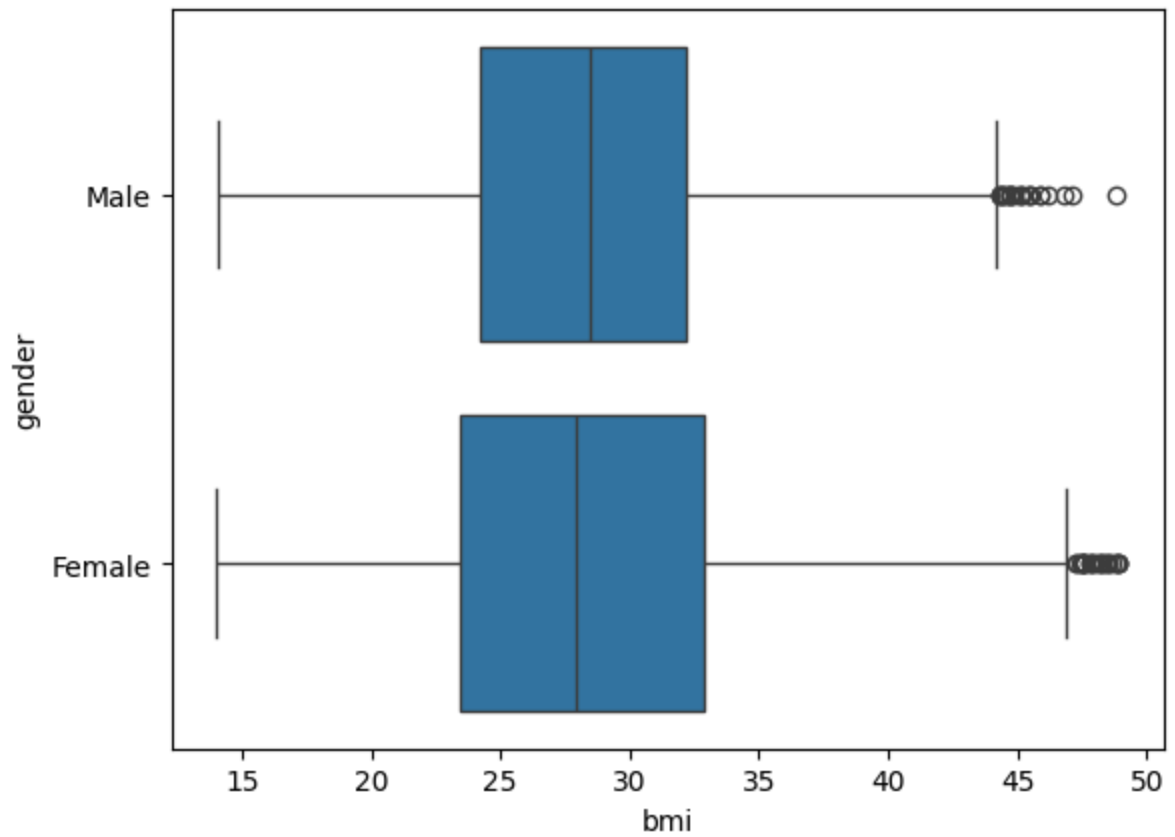
```
In [ ]: sns.boxplot(y='Residence_type', x = 'bmi', data=df)
```

```
Out[ ]: <Axes: xlabel='bmi', ylabel='Residence_type'>
```



```
In [ ]: sns.boxplot(y='gender', x = 'bmi', data=df)
```

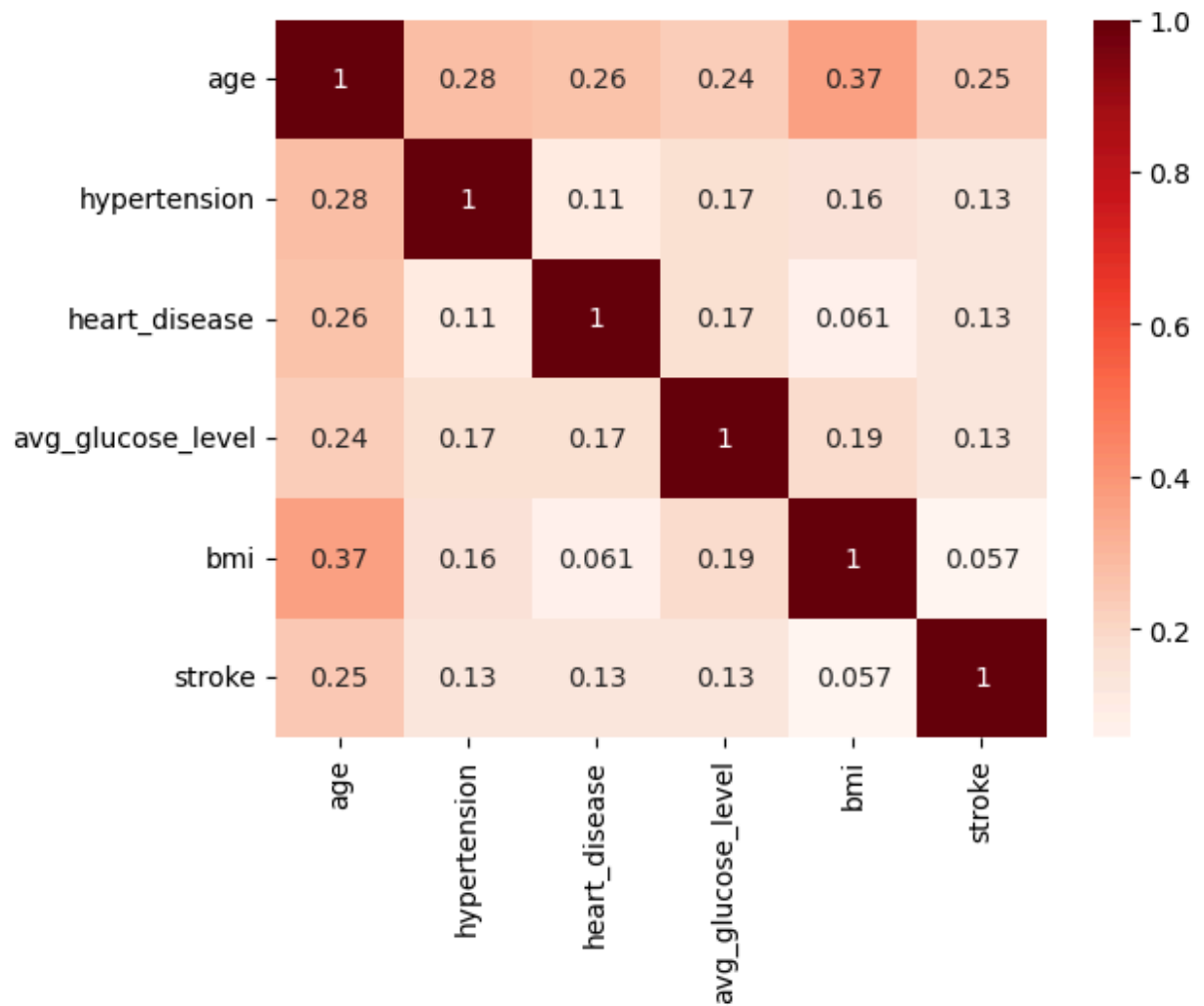
```
Out[ ]: <Axes: xlabel='bmi', ylabel='gender'>
```



```
In [ ]: df.columns
```

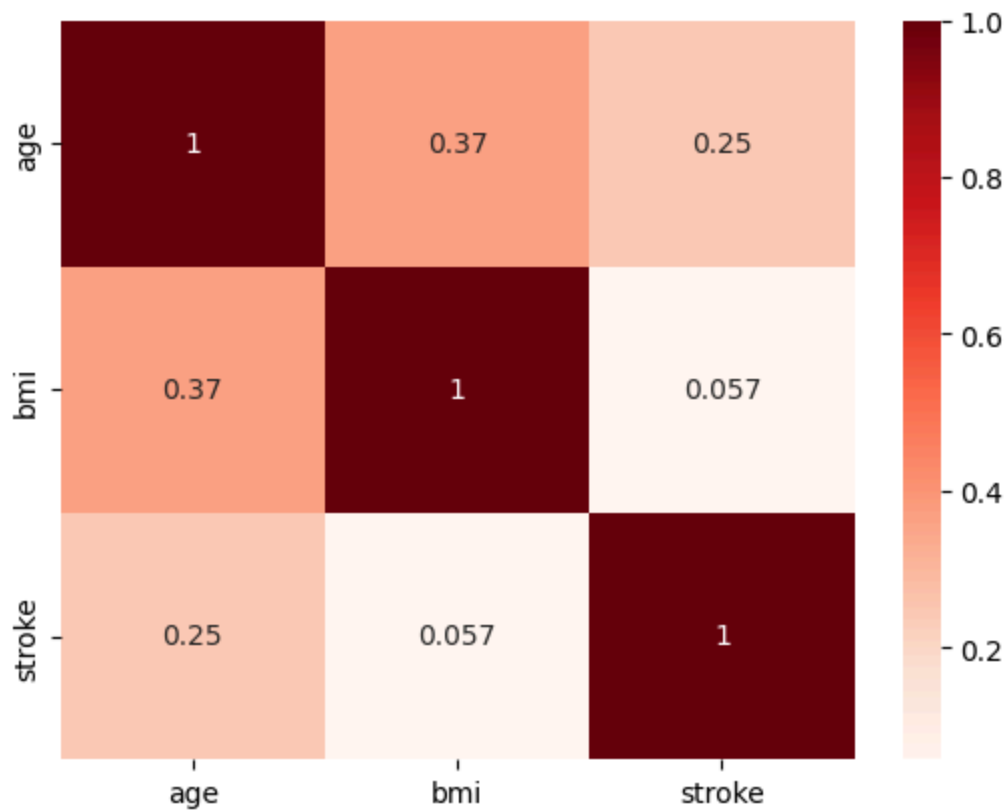
```
In [ ]: # for better uderstanding of correlation plot heatmap of correlation  
sns.heatmap(df.corr(numeric_only = True), cmap = 'Reds',annot = True)
```

```
Out[ ]: <Axes: >
```



```
In [ ]: sns.heatmap(df[['age', 'bmi', 'stroke']].corr(numeric_only = True), annot = True, cma
```

```
Out[ ]: <Axes: >
```



In []: